



École Nationale Supérieure d'Information pour l'Industrie et l'Entreprise

Projet de Logiciel Cluster

”Mise en place d'un supercalculateur dans un centre informatique”

BÉLIER Simon
CHOALER Titouan



7 mai 2026

Table des matières

1	Conformité de la réponse technique au cahier des charges	1
1.1	Partition AC	1
1.2	Partition ABM	2
1.3	Partition AG	3
1.4	Partition LOG	4
1.5	Débit I/O vers un cluster de stockage Lustre	5
1.6	Débit I/O vers le Backbone	5
1.7	Partition Amd/Serv	6
1.8	Offre logicielle	6
2	Nommage du cluster et des composants matériels	7
3	Architecture du réseau de contrôle électrique 1Gb du cluster	8
4	Architecture du réseau d'administration 10 Gb du cluster	11
5	Connectivité externe au Backbone de l'université	12
6	Architecture du réseau d'interconnexion Infiniband	13
7	Extensibilité de la solution	14
8	Récupération des logs et des consoles	14
8.1	Choix des machines et de l'architecture	14
8.2	Fichier de configuration	17
9	Synchronisation temporelle du cluster	20
10	Architecture LDAP	22
10.1	Changements nécessaires côté LDAP universitaire	24
11	Architecture DNS	24
11.1	Conventions de nommage et adresses IP	24
11.2	Configuration et architecture	26
11.3	Changements requis côté DNS de l'université	29
12	Configuration du contrôleur SLURM	30
12.1	Choix des serveurs et installation des dépendances	30
12.2	Configuration Slurm	31
12.3	Restriction de l'allocation des ressources	35
12.4	Vérification de la politique mise en place	36
13	Configuration de la surveillance	36
13.1	Analyse des noeuds de login	37

14 Configuration Puppet	38
15 Environnement Utilisateur	40
16 Déploiement	41
17 Planification de l'installation du cluster	41

1 Conformité de la réponse technique au cahier des charges

1.1 Partition AC

Dans le cahier des charges, il est demandé une partition AmdCompute (AC) de nœuds de calcul d'environ 50 000 cœurs. Or, dans la réponse apportée par la société NextClu, la partition AC comporte au total 43 488 cœurs. En effet, la partition AC, proposé par NextClu compte 302 nœuds, avec par nœud un processeur AMD 9825 ! Et comme un processeur AMD 9825 possède 144 cœurs, on a un total de 144×302 cœurs, à savoir 43 488 cœurs. Ainsi, le nombre de cœurs est en dessous de la demande du cahier des charges ! Et par rapport à la demande, on assiste à une déviation de à peu près -13% sur l'offre, pour le nombre de cœurs de la partition AC ! Ce qui est donc au-dessus de la marge des (+/-)8% acceptés, ... Ainsi, NextClu ne respecte pas le cahier des charges sur ce point-là.

Dans le cahier des charges, il est aussi demandé que chaque nœud de la partition AC aient au moins 2Go de mémoire par cœur. Dans la réponse de NextClu, il est précisé que chaque nœud de la partition AC possède 384 Go de mémoire DDR5. En considérant qu'on a 144 cœurs par nœud avec le choix du processeur associé, on arrive à environ 2.7Go de mémoire DDR5 par cœur ! Ce qui respecte donc la demande sur ce point.

Afin de respecter le cahier des charges, nous proposons une amélioration qui permet un respect total du cahier des charges, et de faire de grandes économies. Cette amélioration consiste à utiliser 240 nœuds pour la partition AC (plutôt que 302 nœuds), et d'utiliser un processeur AMD 9965 (plutôt que AMD 9825) par nœud. Si on suit cette modification, on se retrouve avec exactement 46 000 cœurs dans la partition AC, ce qui représente une divergence de -8% exactement par rapport à la demande ! Ainsi, on respecte le point du cahier des charges qui n'était pas respecté dans la réponse de NextClu. De plus, le deuxième point est toujours respecté, puisqu'avec cette autre proposition, nous avons 2Go de mémoire par cœur précisément !

Cette modification ne fait pas seulement respecter le cahier des charges, mais permet aussi de grandes économies ! En effet, nous avons remarqué que le prix de la partition AC proposé par la société NextClu était de 3 927 812 \$, or le prix de notre proposition pour la partition AC est de 3 555 120 \$! Ce qui représente une économie de 372 692 \$ rien que pour la partition AC avec notre proposition, ce qui est non négligeable ! Cependant, notre proposition diminue notre espace de cache L3 dédié à la partition AC de $(384 \times 62 \text{MB}) = 23\,808 \text{MB}$, et augmente la valeur par défaut de notre enveloppe thermique (TDP) de 2 220 W... L'augmentation de la TDP signifie que notre partition AC va consommer plus d'énergie, et/ou devra nécessiter d'un meilleur refroidissement que la proposition de la société NextClu, ce qui n'est pas inclus dans les coûts directs dans ce projet, mais cela a quand même un effet sur les coûts globaux (donc il ne faut pas le négliger).

Cependant, un point fort qui découle de la proposition de la société est grande capacité de

cache L3 qu'elle propose. En effet, avec notre provoque une diminution de l'espace cache L3 de la partition! Or, l'espace cache L3 est très importantes pour les partitions de calcul, comme la partition AC!...

1.2 Partition ABM

Dans le cahier des charges, il est précisé que la partition AmdBigMem (abrégée ABM dans la suite) doit être équipée d'environ 4000 cœurs de calcul. La solution proposée par la société NextClu comprend 3600 cœurs, ce qui représente une déviation de 10 % par rapport à l'exigence initiale. Bien que cette légère sous-estimation reste relativement acceptable dans certains contextes, elle pourrait poser un problème si les charges de travail futures exigent une capacité de calcul strictement conforme au cahier des charges. Par ailleurs, l'université impose un ratio de 6 Go de mémoire vive par cœur. À ce niveau, la solution de NextClu est conforme, avec approximativement 7 Go de mémoire DDR5 par cœur, ce qui constitue un léger excès favorable, garantissant un confort de travail pour les applications gourmandes en mémoire. Nous proposons une alternative qui permettrait de répondre strictement aux besoins exprimés tout en réduisant le coût global de l'infrastructure, et en améliorant les performances par cœur via un meilleur hyperthreading. Notre proposition repose sur l'utilisation de processeurs AMD EPYC 9655, chacun disposant de 96 cœurs physiques et atteignant une fréquence maximale de 4.5 GHz, ce qui est particulièrement intéressant pour les charges intensives et les traitements massivement parallèles. Grâce à ces processeurs, il serait possible d'atteindre les 4000 cœurs requis en déployant uniquement 42 nœuds de calcul, contre un nombre probablement plus élevé dans la solution de NextClu. Ce dimensionnement permet de contenir le coût de l'architecture à 485 932 €, contre 524 300 € pour la solution initiale, soit une économie directe de 38 368 €.

Cette configuration alternative repose sur un nombre de cœurs plus élevé par nœud, ce qui induit un ratio mémoire par cœur légèrement inférieur à celui requis. En effet, avec 512 Go de RAM par nœud, on obtient environ 5.3 Go de mémoire par cœur, ce qui reste en dessous du seuil demandé de 6 Go. Pour remédier à cela et assurer une pleine conformité au cahier des charges, il suffirait d'augmenter la mémoire à 572 Go par nœud, ce qui implique l'ajout d'approximativement 60 Go de RAM supplémentaires par machine. Sur 42 nœuds, cela représente un total de 336 Go de mémoire DDR5 à ajouter à la configuration globale. L'achat de cette mémoire supplémentaire reste économiquement viable tant que son coût ne dépasse pas les 38 368 € économisés sur le reste de l'infrastructure. Cette approche permettrait donc à la fois de respecter intégralement les spécifications techniques du cahier des charges et de réduire le coût global du projet, tout en tirant parti des meilleures caractéristiques techniques offertes par les processeurs EPYC 9655, notamment en matière d'hyperthreading, de cache L3 et de fréquence boost.

Voyons à présent le racking de cette partition. La solution de NextClu pour la partition AmdBigMem (ABM) repose sur l'intégration des lames NxC-B30 dans des châssis NxC-C30, chaque châssis pouvant contenir jusqu'à 30 lames. Chaque châssis occupe 16U dans un rack de 48U, et un seul rack est nécessaire pour héberger la partition ABM (nommé rkABM[1-1]). Ce rack est dédié à la partition ABM et contient toutes les lames nécessaires au calcul de

cette partition. Les lames sont interconnectées via des ports Ethernet 100Mb (pour la gestion IPMI), Ethernet 1Gb (pour l'administration) et Infiniband HDR200 (pour la communication interne à haute vitesse). Ces lames sont reliées à des switchs Infiniband HDR200, intégrés dans le fond de panier du châssis, ainsi que des switchs Ethernet pour la gestion et le contrôle. Cela permet une interconnexion rapide et une gestion efficace du cluster. Un des avantages de cette solution est la gestion de l'alimentation via des alimentations redondantes et des contrôleurs intelligents pilotables par Ethernet qui assurent une haute fiabilité de l'alimentation.

En termes de gestion des ressources, l'architecture est optimisée pour la densité de nœuds et la connectivité avec un seul rack pour la partition ABM, permettant ainsi une utilisation efficace de l'espace. Toutefois, le nombre de cœurs proposé dans cette partition est de 3600 cœurs, soit une déviation de 10 % par rapport aux 4000 cœurs demandés, ce qui pourrait poser un problème dans le futur si les exigences de performance augmentent...

1.3 Partition AG

Pour la partition AmdGpu (AG), il est demandé dans le cahier des charges d'avoir environ 4 000 cœurs avec au moins 2Go de mémoire par cœur. Dans la réponse technique de la société, il est proposé que la partition AG compte 50 nœuds, et que pour chaque nœud, on ait un processeur AMD 9655, avec 256 Go de mémoire DDR5.

En termes de nombre de cœurs, la partition AG proposé par la société comporte ($96 \times 50 =$) 4 800 cœurs au total. Ce qui représente une divergence de +20% par rapport à la demande originelle. Ainsi, cette proposition n'est pas acceptable, car elle ne respecte pas la demande de l'université en termes de nombre de cœurs (divergence de 8

En termes d'espace mémoire par cœur, la proposition de la société implique qu'on ait ($256\text{Go}/96 =$) 2.7Go de mémoire par cœur! On est au-dessus des 2Go de mémoire par cœur demandé par l'université, donc ce point est respecté.

Le cahier des charges demande aussi que la partition AG permette d'exécuter des codes CUDA sur les cartes NVIDIA et/ou d'utiliser des solutions de visualisation à distance. Et que la partition AC doit offrir l'accès à au moins 50 cartes.

La réponse de la société NextClu propose justement 1 carte GPU Nvidia A100 par nœud, et comme dans cette proposition, on a 50 nœuds, on a donc un total de 50 cartes GPU Nvidia A100 et l'exécution de codes CUDA sur les cartes NVIDIA est possible, ce qui respecte le cahier des charges!

En supposant qu'on ne puisse mettre qu'une carte GPU Nvidia par processeur (par nœud), on va essayer de chercher une meilleure proposition de partition en 50 nœuds, et qui a un nombre de cœurs entre 3 680 et 4 320 cœurs (afin de ne pas dépasser une divergence de plus de 8% par rapport à la demande). Un processeur idéal pour une telle solution aura donc entre 73.6 et 86.4 cœurs! Ce qui est impossible avec l'hypothèse qu'on ait qu'une carte GPU NVIDIA A100 par processeur! En effet, car aucun processeur a un nombre de cœurs entre 73.6 et 86.4 cœurs! Les plus proches sont le processeur AMD 9565 avec 72 cœurs, le processeur AMD 9645 et AMD 9655 avec 96 cœurs. Ainsi, on brise l'hypothèse précédente,

et on suppose désormais qu'on met 2 cartes GPU NVIDIA A100 par processeur (par nœud). Ainsi, on va chercher une meilleure proposition de partition AG avec 25 nœuds, et qui a un nombre de cœurs entre 3 680 et 4 320 cœurs. Un tel processeur aura donc entre 147.2 et 172.8 cœurs! Ce qui est parfait, car il existe qu'un seul processeur parmi ceux représentés qui le permet, c'est le processeur AMD 9845 avec 160 cœurs!

Notre proposition d'amélioration consiste ainsi à utiliser une partition AG comptant 25 nœuds (à la place de 50), et avec par nœud un processeur AMD 9845 (plutôt que AMD 9655), et 2 cartes GPU Nvidia A100 (plutôt que une seule carte)! Afin de continuer à respecter l'espace mémoire minimum par cœur, on va devoir monter les 256Go de mémoire par nœud de la proposition de NextClu, en 320Go de mémoire par nœud!

Cette proposition d'amélioration permet de respecter le nombre de cœurs dans la partition AG. En effet, on a dans ce cas-là exactement 4 000 cœurs. De plus, la quantité d'espace mémoire par cœur est aussi précisément de 2 Go, et le nombre de cartes GPU Nvidia A100 est toujours respecté (on en a 50)! La partition que nous proposons permet aussi d'exécuter du code CUDA sur les cartes NVIDIA, et donc respecte entièrement le cahier des charges!

En ce qui concerne le prix, notre proposition nous permet aussi de faire des économies au niveau des processeurs de la partition! L'ancienne proposition consistait en 50 processeurs AMD 9655, ce qui revient à un prix de 592 600 \$, alors que notre proposition qui consiste en 25 processeurs AMD 9845 revient à un prix de 339 100 \$. Ainsi, notre proposition économise 253 500 \$ par rapport à la proposition de la société! De plus, notre proposition diminue l'enveloppe thermique (TDP) totale de la partition AG de 8 405 W par rapport à la proposition de la société, ce qui veut dire d'une certaine manière qu'on risque de consommer moins d'énergie, ce qui fait des économies aussi sur la durée. Cependant, avec notre proposition, la mémoire cache L3 totale de la partition est réduite de 11 200 MB par rapport à la proposition de la société. Ce qui peut être néfaste au niveau performance, car on se trouve sur une partition de calcul.

1.4 Partition LOG

La société NextClu propose 10 nœuds de login pour la partition Login (LOG), ce qui est assez pour séparer les différentes communautés d'utilisateurs et leur offrir suffisamment de puissance de calcul interactive pour développer, compiler et déverminer leurs applications, comme ce qui est demandé par le cahier des charges, si le reste de la structure des nœuds est bien faite.

D'après le cahier des charges, chaque nœud de la partition LOG doit comporter 32 cœurs, 2 Go de mémoire par et 2 disques sécurisés pour le système avec espace temporaire (/tmp) d'au moins 6 To au total. En ce qui concerne le nombre de cœurs, la proposition de la société ne respecte pas du tout le cahier des charges. En effet, la société utilise un processeur AMD 9535 pour chacun de ses nœuds, et ce processeur est constitué de 64 cœurs, et non 32! La réponse de la société laisse $(96 \text{ Go}/64 =) 1.5 \text{ Go}$ de mémoire DDR5 par cœur, ce qui ne respecte pas non plus le cahier des charges, car il était demandé 2 Go de mémoire par cœur spécifiquement. Au tour des disques sécurisés pour le système, la proposition de la société

considère 2 disques de 4 To pour chaque nœud, ce qui revient au total à 8 To. Cela répond au cahier des charges, car on a plus de 6 To en considérant les 2 disques.

Afin de répondre au cahier des charges, nous proposons de modifier le processeur utilisé pour les nœuds de la partition LOG, et aussi la taille de l'espace mémoire DDR5. Comme processeur, nous prenons le processeur AMD 9335, car il possède 32 cœurs, et permet de faire plus d'économies que le processeur AMD 9355 (aussi à 32 cœurs) au prix d'une cache L3 moins importante, et permet aussi de respecter le cahier des charges. Nous avons sacrifié ici de la cache L3, car la mémoire cache L3 est beaucoup moins importante pour les nœuds de login ! Pour la mémoire DDR5, nous proposons de passer de 96 Go à 64 Go, de telle sorte que chaque cœur à 2 Go, comme ce qui était demandé sur le cahier des charges.

Notre proposition permet ainsi de réaliser des économies de 58 140 \$, tout en respectant le cahier des charges. Cette modification par rapport à la proposition de la société comporte une baisse de la mémoire cache L3 de telle sorte, qu'on ait 2 fois moins de mémoire cache L3 par rapport à la société. Mais on a jugé la mémoire cache L3 peu importante pour les nœuds login, donc cette baisse n'est pas critique. Aussi, on a une baisse de l'enveloppe thermique de 900 W par rapport à la proposition de la société, ce qui implique des économies sur le long terme sur le coût de maintien de la partition LOG.

1.5 Débit I/O vers un cluster de stockage Lustre

La société NextClu a prévu de mutualiser les fonctions de routeurs Lustre et routeurs Backbone Ethernet sur les 6 nœuds routeurs I/O. Pour la fonction Lustre, ils seront connectés sur deux ports IB HDR200, l'un relié sur le futur réseau de stockage, et l'autre sur le réseau d'interconnexion du cluster de calcul.

Chaque routeur, parmi les 6, aura un débit réel de 24 Go/s. Ce qui, en parallèle, fait donc un débit I/O global de 144 Go/s ! Ce qui est loin en dessous de la valeur demandée originellement par l'université, sur le cahier des charges : "au moins 180 Go/s". Ainsi, le cahier des charges n'est pas respecté par la société sur ce point, et notre proposition devra améliorer ce point. Autrement, les cartes Infiniband proposées par la société, sont des cartes ConnectX6 Mellanow HDR200 200Gbs. Ce qui correspond à la demande de l'université qui souhaitait que son réseau de stockage Lustre soit un réseau Infiniband HDR 200Gbs.

1.6 Débit I/O vers le Backbone

L'université demande un débit I/O global d'au moins 25 Gbits/s (donc d'au moins 3.1 Go/s) vers le Backbone pour l'ensemble des nœuds des 3 partitions de calcul : AC, ABM, et AG. Dans sa proposition, la société NextClu, connectera chacun des 6 nœuds routeurs par 2 ports 10Gb au backbone Ethernet de l'université. Or, chaque port 10Gb Ethernet à un débit réel de 1 Go/s (= 8 Gbits/s). Comme on a 6 nœuds routeurs I/O, on a alors 12 ports 10Gb Ethernet au total, ce qui fait donc un débit total de 96 Gbits/s ! Ce qui est nettement supérieur au 25 Gbits/s requis. Ainsi, le cahier des charges est respecté.

1.7 Partition Amd/Serv

Dans le cahier des charges, il est demandé par l'université que la partition Amd/Serv contienne au moins 10 nœuds de services avec 16 cœurs et 256 Go de mémoire par nœud, ainsi que 2 disques de 1 To pour le système et 2 disques de 4 To pour les données du système. La solution apportée par la société NextClu propose 14 nœuds, ce qui dépasse les exigences en plus d'apporter une plus-value en termes de redondance et de répartition des charges. Cependant, les nœuds maîtres et de services du cluster disposent de 2 disques de 3 To au lieu des 4 To demandés, et les nœuds routeurs I/O ne disposent que de 2 disques de 1 To.

Nous proposons une solution optimisée afin de respecter pleinement le cahier des charges sans engendrer de surcoût significatif. Notre alternative consiste à remplacer les disques de 3 To des nœuds maîtres et de service cluster par des disques de 4 To, ce qui permet de répondre exactement aux besoins de stockage demandés. Concernant les nœuds routeurs I/O, une mise à niveau vers 2 disques de 4 To pourrait être envisagée.

Cette solution garantirait une parfaite conformité avec le cahier des charges tout en optimisant les ressources sans dépasser le budget initial significativement.

Nous pouvons néanmoins noter des points forts significatifs vis-à-vis de la solution de NextClu : Le nombre de nœuds de services

Le cahier des charges demande au moins 10 nœuds, la réponse en propose 14 (2 nœuds maîtres, 6 nœuds de service cluster, 6 nœuds routeurs I/O), ce qui est une plus-value en termes de redondance et de répartition des charges.

Conformité CPU mémoire Chaque nœud est équipé de 2 processeurs AMD 9255 et de 256 Go de mémoire DDR5, conformes aux exigences minimales.

Conformité connectivité Les nœuds disposent des interfaces réseau requises, avec des ports Infiniband HDR200 et Ethernet 1Gb/s et 10Gb.

Usage des VLANS L'usage de VLANs pour compartimenter les réseaux est une bonne pratique qui semble compatible avec l'architecture actuelle du centre informatique.

Pour la partition Amd/Serv, NextClu a prévu un total de 2 racks (nommés rkSrv[1-2]) pour héberger les nœuds de service, y compris les serveurs d'administration, les routeurs, les serveurs de login et le serveur NFS. Ces racks de 48U hébergent non seulement les serveurs, mais aussi les switchs Ethernet pour la gestion et la connectivité du cluster. Le rack des serveurs inclut également des switchs Ethernet et Infiniband pour assurer une gestion efficace des communications internes et externes. Le nombre de nœuds de service proposé dépasse la demande du cahier des charges (avec 14 nœuds au lieu de 10), offrant ainsi une plus-value en termes de répartition de la charge et de redondance, ce qui est bénéfique pour la disponibilité du cluster.

1.8 Offre logicielle

L'offre logicielle proposée par la société comprend une licence RHEL 9.3, ce qui répond au cahier des charges de l'université ! De plus, l'offre logicielle proposée comprend aussi les

rpms Lustre pour les nœuds clients et routeurs, des compilateurs Intel et AMD, et ce même si uniquement des processeurs AMD sont utilisés). Le compilateur Intel a sûrement pour fonction d'assurer la compatibilité avec du code assembleur Intel, si besoin (car sinon il est inutile). L'offre logicielle est aussi constituée d'un débogueur DDT, et de logiciels Open Source. Ainsi, les outils de développement et de profiling de la proposition sont adaptés aux processeurs sélectionnés (sauf le compilateur Intel), et donc répondent au cahier des charges. Cependant, le cahier des charges souhaité un debugger TotalView, alors que celui utilisé dans la proposition est le debugger DDT (Dynamic Debugging Technique), donc la proposition d'offre logiciel ne répond pas totalement au cahier des charges. Toutefois, le debugger DDT est en réalité très similaire au debugger TotalView au niveau de ses fonctionnalités, et même plus facile d'utilisation pour un utilisateur débutant ! En effet, l'interface de DDT est souvent considéré comme plus fluide et facile d'utilisation que l'interface de TotalView, et donc, même si la société ne répond pas totalement au cahier des charges en incluant le debugger DDT, plutôt que TotalView, le choix n'est pas dénué de sens. Si l'université souhaite seulement utiliser les fonctionnalités de base de TotalView, alors le choix proposé par la société, DDT peut être meilleure alternative.

2 Nommage du cluster et des composants matériels

Nom du cluster et organisation logique des nœuds

Le nom du cluster doit être simple à écrire et à retenir dans une utilisation quotidienne, car les administrateurs vont fréquemment devoir le taper au clavier. Ainsi, étant donné que l'acronyme de l'Université de Grenoble-Alpes est UGA, nous choisirons comme nom pour le cluster "uga".

Au niveau de l'organisation logique des nœuds, nous les numérotions comme ceci : ugaXXX avec XXX le numéro du nœud. Le numéro des nœuds sera choisi conformément à leur appartenance dans leur groupe spécifique (voir partie suivante).

Définition des groupes clustershell

En m'inspirant du fichier cluster.yaml de l'ENSIIE et de la structure décrite, voici une proposition de configuration pour le fichier /etc/clustershell/groups.d/cluster.yaml adaptée à la configuration du cluster de l'université :

Nous avons laissé entre chaque nodeset de la place pour ajouter des serveurs aux partitions existantes. Par exemple, si l'on voulait ajouter un serveur nfs il serait numéroté uga61. Ou bien si l'on veut ajouter 100 nœuds de calcul pour la partition AC, ils seront numérotés uga[602-701].

Nom du groupe @xxxx	Nodeset	Commentaire
@login	uga[000-009]	Nœuds de login où les utilisateurs se connectent au cluster.
@master	uga[020-021]	Nœuds maîtres pour la gestion et l'administration du cluster.
@services	uga[030-035]	Nœuds dédiés aux services (serveur NFS, services divers, etc.).
@io	uga[050-055]	Nœuds routeurs gérant l'interconnexion des partitions de calcul.
@nfs	uga[060]	Serveur NFS pour la gestion du stockage partagé du cluster.
@amd_big_mem	uga[100-149]	Nœuds de calcul des partitions AMB.
@gpu_compute	uga[200-249]	Nœuds de calcul de la partition AG avec GPU pour le calcul parallèle.
@amd_compute	uga[300-601]	Nœuds de calcul des partitions AC.
@all	uga[000-601]	Tous les nœuds du cluster, pour exécuter des commandes globales.

TABLE 1 – fichier /etc/clusterhell/groups.d/cluster.yaml

3 Architecture du réseau de contrôle électrique 1Gb du cluster

Utilisation des ports des switchs : Switch 1 : 33 ports utilisés, 7 de libre Switch 2 : 36 ports utilisés, 4 de libre Switch 3 : 34 ports utilisés, 6 de libre

Il y a donc 17 ports de libre et qui peuvent être utilisés dans le cas d'une panne sur d'autres ports. Ainsi, nous avons supprimé la majorité des SPOFs et augmenté la résilience en ayant des ports de secours.

Le réseau de contrôle voit passer des connexions IPMI, SNMP, Ethernet 1GB et des gestions de l'alimentation (PDU).

Le logiciel nécessaire pour unifier les commandes de contrôle électrique est IPMI couplé avec un système SNMP pour la gestion réseaux et des PDUs.

Le courant soutenu fait référence à une alimentation électrique ininterrompue, garantissant une source d'énergie continue en cas de panne du réseau principal grâce à des systèmes comme les UPS ou les générateurs de secours. Si l'université dispose de ce type de courant, il est crucial de connecter les équipements critiques du cluster informatique, tels que les serveurs principaux (notamment les nœuds maîtres et les nœuds de calcul), les switchs réseau (Ethernet et Infiniband), les nœuds de stockage (comme le serveur NFS) ainsi que les systèmes de contrôle électrique (comme les PDUs et les UPS eux-mêmes) à ce réseau électrique soutenu. Cela permet de garantir la continuité des opérations en cas de coupure

Noeuds	Port Switch NxC-W40- 1g	Commentaire
nœud master1 ports Gb [1-2]	swc[1-2]p1	Élimination des SPOFs en connectant un port sur chaque swicths
nœud master2 ports Gb [1-2]	swc2p2 swc3p1	,
1 serveur NFS admin, port BMC	swc1p2	
6 nœuds de service, port BMC	swc1p[3-4], swc2p[3-4], swc3p[2-3]	On répartit les nœuds de services sur les 3 switchs (2 pour chaque)
10 nœuds de login, port BMC	swc1p[5-6], swc2p[5-8], swc3p[4-7]	On répartit les nœuds de login sur les switch, en n'en mettant pas trop sur le premier qui accueille les masters.
6 nœuds routeur, port BMC	swc1p[7-8], swc2p[9-10] , swc3p[8-9]	On répartit les nœuds de services sur les 3 switchs (2 pour chaque)
50 nœuds graphiques, port BMC	swc1p[9-25], swc2p[11-27] , swc3p[10-25]	On répartit les nœuds de services sur les 3 switchs (17, 17 et 16 pour chaque)
Châssis NxC-C30, port BMC	swc1p[26-33] , swc2p[28-35], swc3p[26-33]	11 châssis pour la partition AC et 1 pour la partition ABM à raison de 2 ports BMC par châssis. Le tout faisant 24 ports répartis équitablement.
Alimentation électrique (PDU)	swc2p[36], swc3p34	

TABLE 2 – Architecture du réseau de contrôle électrique 1Gb du cluster

de courant et de protéger les équipements sensibles.

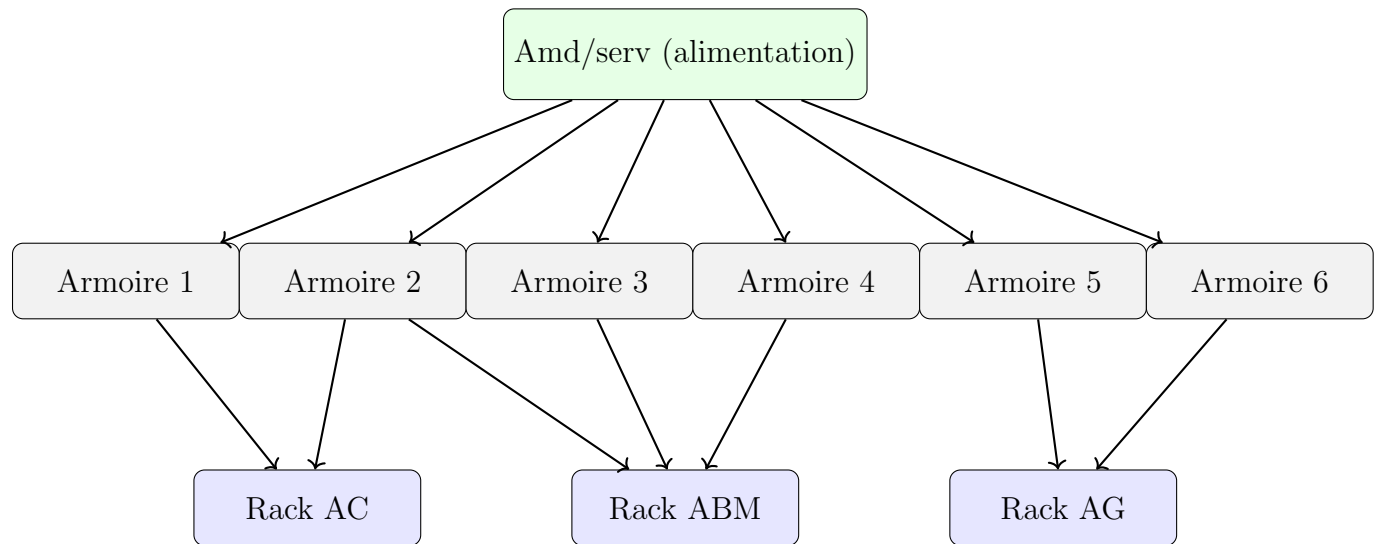


FIGURE 1 – Branchement électrique du système d'alimentation aux racks

4 Architecture du réseau d'administration 10 Gb du cluster

Noeuds	Port Switch NxC-W40-1g	Commentaire
nœud master1 ports 10Gb [1-2]	swa[1-2]p1	Élimination des SPOFs en connectant un port sur plusieurs switches (un maximum)
nœud master2 ports 10Gb [1-2]	swa[3-4]p1	Idem
1 serveur NFS adm, 4 port 10Gb	swa[1-4]p2	Idem (On aurait pu mettre 1 port du cinquième switch à la place d'un port d'un autre switch. Mais comme cela ne change pas grand-chose pour la suite, et pour la propreté, j'ai préféré ne pas le mettre)
6 nœuds de service, 1 port 10Gb	swa[1-4]p3, swa5p[1-2]	Idem
10 nœuds de login, 1 port 10Gb	swa[1-4]p[4-5], swa5p[3-4]	Idem
6 nœuds routeur, 1 port 10Gb	swa[1-4]p6, swa5p[5-6]	Idem
50 nœuds graphiques, 1 port 10Gb	swa[1-5]p[7-16]	Idem + On répartit équitablement sur les 5 switches les nœuds
Châssis NxC-C30, 3 port 10Gb	swa[1-4]p[17-23],swa[5]p[17-24]	Idem + On a 3 port 10Gb, et 12 châssis (10 pour uniquement AC, 1 pour uniquement ABM, et 1 pour les nœuds restants de AC et ABM). Donc, on doit répartir le plus équitablement possible 36 ports. La répartition faite est (7,7,7,7,8).

TABLE 3 – Architecture du réseau d'administration 10 Gb du cluster

Utilisation des ports des switches : Swicths 1, 2, 3 et 4 chacun : 23 ports utilisés, 17 ports restants
Switch 5 : 24 ports utilisés, 16 ports restants.

Ainsi, on énumère un total de 84 ports libres sur le réseau d'administration.

Dans le cas de la panne d'un port avec l'architecture proposé dans le tableau modifié, les nœuds connectés à plusieurs ports 10Gb ne sont pas affectés, et restent totalement fonction-

nels. En ce qui concerne les nœuds mono-connectés, lors de la panne d'un port on assiste à une perte partielle du service, mais comme la charge est répartie, on évite une interruption globale du service! Dans le cas de la panne d'un switch, environ 23 24 connexions sont perdues, mais grâce à la répartition uniforme utilisée, aucune catégorie de nœuds n'est entièrement indisponible! Donc le cluster continue à fonctionner, mais cependant, l'administration des services du cluster est réduite de 20%. Que cela soit juste la panne d'un port, ou la panne d'un switch, les 84 ports libres restants sont suffisant pour servir de ports/liaisons de secours/remplacement! Ce qui permet d'améliorer d'autant plus la résilience du réseau.

Les types de flux, protocoles et services qui passent sur le réseau d'administration sont ceux qui sont liées à l'installation des nœuds, et/ou aux services d'administration (ex : gérer le cluster : SSH, Puppet,..., le stockage : NFS,..., l'authentification : LDAP,..., etc,...).

5 Connectivité externe au Backbone de l'université

Les nœuds de services du cluster qu'il faut connecter au réseau Backbone de l'université, sont les 2 nœuds maîtres, les 6 nœuds de services, les 6 nœuds de routeurs I/O, et les 10 nœuds de login. Étudions chacun de ces catégories de nœuds :

Les nœuds maîtres sont utilisés pour permettre le déploiement du cluster, et aux gérants du cluster, de s'y connecter en tant qu'administrateur et d'avoir accès à tous les services disponibles. Les nœuds de service sont utilisés pour héberger les services internes nécessaires à son administration. Les nœuds routeurs I/O sont utilisés pour la transmission d'information des nœuds de calculs avec le réseau backbone et le réseau de stockage Lustre. Les nœuds login sont utilisés pour permettre la connexion au cluster, des utilisateurs de l'université.

Essayons de prévoir le nombre de connexions 10Gb au backbone qu'on doit demander à l'équipe réseau, pour ces 4 catégories de nœuds : On a 2 nœuds maîtres, et chacun de ces nœuds ont 4 ports 10 Gb/s. Mais, parmi ces 4 ports, 2 sont déjà sur le réseau d'administration, donc il nous en reste 2. On utilise les 2 ports restants sur le réseau backbone. On prévoit donc 4 connexions 10 Gb au backbone pour les nœuds maîtres. On a 6 nœuds de service, et chacun de ces nœuds ont 4 ports 10 Gb/s. Mais parmi ces 4 ports, 1 est déjà sur le réseau d'administration, donc il nous en reste 3. Cependant, afin d'améliorer la résilience sur le réseau d'administration, on peut accorder un deuxième port au réseau d'administration, afin de ne pas avoir de nœuds mono-connectés pour cette catégorie. Ainsi, il nous reste dans ce cas 2 ports 10 Gb, qu'on utilise sur le réseau backbone. On prévoit ainsi 12 connexions 10 Gb au backbone pour les nœuds de service. On a 6 nœuds routeurs I/O, et chacun de ces nœuds ont 6 ports 10 Gb/s. Mais parmi ces 6 ports, 1 est déjà sur le réseau d'administration, donc il nous en reste 5. Cependant, afin d'améliorer la résilience sur le réseau d'administration, on peut accorder un deuxième port au réseau d'administration, afin de ne pas avoir de nœuds mono-connectés pour cette catégorie. Ainsi, il nous reste dans ce cas 4 ports 10 Gb, qu'on utilise sur le réseau backbone. On prévoit ainsi 24 connexions 10 Gb au backbone pour les nœuds routeurs I/O. On a 10 nœuds login, et chacun de ces nœuds ont 2 ports 10 Gb/s. Mais, parmi ces 2 ports, 1 port est déjà sur le réseau d'administration, donc il nous en reste qu'un seul. On utilise le port restant sur le réseau backbone. On prévoit donc 10 connexions 10 Gb au backbone pour les nœuds login.

On prévoit ainsi de demander 50 connexions 10Gb au backbone, à l'équipe réseau.

Nodeset	Serveurs université	Flux/protocoles
@login	la-sure[1-2], la-buffe[1-2], izon, stations (Bureaux professeurs et PME, salles TPs)	DNS, LDAP, NFS, SSH
@admin	la-sure[1-2], la-buffe[1-2], charande[1-2], rachais[1-2], izon, stations (Bureaux professeurs et PME, salles TPs)	DNS, LDAP, NFS, NTP, PXE, SSH
@services	La-buffe[1-2], charande[1-2], rachais[1-2]	DNS, NTP, PXE
@io	La-sure[1-2]	DNS, NFS

TABLE 4 – Connectivité externe au Backbone de l'université

6 Architecture du réseau d'interconnexion Infiniband

Noeuds	Port Infiniband	Flux/Commentaire
Noeuds master x 2	sw1n[1-2]p1	Redondance sur 2 switches L1 pour éviter SPOF.
Noeuds de services x 6	sw1n[1-3]p[2-3]	Répartition sur 3 switches L1 pour résilience.
Noeuds de routeur x 6	sw1n[1-3]p[3-4]	Répartition de charge, 3 switches L1, redondance.
Noeuds de login x 3, ugp[0-2]	sw1n[1-3]p20	Redondance
Noeuds de login x 2, ugp[3-4]	sw1n[2-3]p21	Redondance
Noeuds de login x 3, ugp[5-7]	sw1n[1-3]p22	Redondance
Noeuds de login x 2, ugp[8-9]	sw1n1,3p23	Redondance

TABLE 5 – Architecture du réseau d'interconnexion Infiniband

L'architecture du réseau d'interconnexion InfiniBand du cluster est basée sur une topologie Fat Tree avec 6 switches L2 (Top switches) et 17 switches L1 (Leaf switches). Les switches L1 sont connectés à jusqu'à 30 nœuds, tandis que les switches L2 sont interconnectés entre eux et reliés aux switches L1, formant une structure résiliente. Les partitions AC, AL et AX sont desservies par des switches L1 dédiés, tandis que les nœuds de service et login sont répartis sur d'autres switches L1 dédiés. Cette architecture permet une répartition équilibrée de la charge, avec une redondance et une résilience accrues, limitant les risques de SPOFs et garantissant une connectivité continue pour les nœuds du cluster, y compris les nœuds de routeur et les nœuds maîtres. Chaque nœud, à l'exception du serveur NFS, est équipé d'un port InfiniBand pour se connecter à ce réseau haute performance.

7 Extensibilité de la solution

Les 302 nœuds de la partition AC sont répartis dans 10 châssis pouvant accueillir 30 lames. Pour ce qui est des 2 derniers, ils sont inclus dans l'espace restant du châssis non remplis de la partition ABM. Cette dernière est constituée de 50 nœuds répartis dans 2 châssis. Il y a donc 1 châssis avec 20 nœuds ABM et les 2 nœuds AC restants. Il y a donc de la place pour ajouter 8 nœuds de calcul dans le châssis non remplis de la partition ABM. Ainsi, il est possible d'ajouter 8 nœuds sans ajouter de châssis.

Il est toutefois possible d'ajouter des châssis sans augmenter le nombre de racks. Chaque rack de 48U peut accueillir 3 châssis NC-C30 de 16U. La partition AC occupe actuellement 4 racks avec 10 châssis d'installés sur une capacité totale de 12 racks. Ce qui permet d'ajouter 2 châssis supplémentaires, soit 60 nœuds de calcul. Pour la partition ABM, un seul rack est utilisé avec 2 châssis installés sur une capacité de 3, laissant la possibilité d'ajouter 1 châssis, soit 30 nœuds.

On a plusieurs facteurs limitant pour étendre la partition AG : Premièrement, la place prise par un nœud de la partition AG. En effet, un nœud de la partition AG occupe 2U, ce qui est deux fois plus qu'un nœud de la partition AC, ou ABM. Aussi, les nœuds AG sont connectés à des switches IB L1 dédiés. L'extension de la partie AG, nécessiterait aussi une augmentation du nombre de ports sur ces switches, ou voir l'ajout de nouveaux switches L1 ! L'augmentation du nombre de switches L1 nécessiterait aussi une augmentation de switches L2. Or, ces switches prennent aussi de l'espace sur le rack rkW. ...

Pour étendre la partition AG, on peut ajouter des racks et étendre le réseau IB.

Les autres facteurs limitant l'extensibilité du cluster sont le débit I/O global vers le cluster de stockage Lustre, et vers le backbone, qui sont respectivement fixés à 144 Go/s et 96 Gb/s (d'après l'analyse de la réponse). En effet, une extension du cluster pourrait possiblement saturer la bande passante de ces débits,... Un autre facteur pourrait être l'augmentation de la production de la production de chaleur, dû à cette extension.

8 Récupération des logs et des consoles

8.1 Choix des machines et de l'architecture

L'objectif est de mettre en place une récupération des logs des partitions AC, ABM, AG, logins et services. Nous mettrons donc en place un serveur de logs centralisé dans lequel les fichiers de journaux seront organisés dans une arborescence permettant de facilement différencier les différentes machine.

Ils seront stocké dans le répertoire : `/var/cluster/logs/<Hostname>/messages`.

Pour rester scalable nous décidons de stocker les logs au sein du server NFS, sans y faire tourner de démon rsyslog. En effet, si nous décidions de le faire, il y aurait un SPOF computationnel sur ce dernier. Ainsi, pour ne pas reposer sur les capacités du serveur NFS que nous ne connaissons pas, nous déléguons la tâche et la politique de décisions aux noeuds de services.

Les noeuds de services vont gérer la politique de décision et les logrotate. Les logs générés par les noeuds du réseau IB seront envoyés aux noeuds de services (via le réseau IB) puis seront stockés dans le serveur NFS en passant par le réseau d'administration. Le but est de ne pas surcharger le réseau d'administration donc l'idée de passer par le réseau IB nous paraît la plus pertinente. Étant donné que le serveur NFS est relié à 4 switch du réseau d'administration en ethernet 10Gb, nous confions la tâche à 4 noeuds de services : uga[30-33] . Les 4 noeuds auront la même configuration afin de permettre de la redondance et de rajouter de la résilience (principe de failover). Les répertoires de travail `/var/cluster/logs/<Hostname>/messages/` et `/var/cluster/logs/console` seront situés sur un stockage NFS monté en écriture sur les noeuds de services. Ainsi, les deux noeuds maîtres auront bien accès aux logs.

Un problème se pose cependant pour les logrotate. Étant donné que les rotations sont effectuées à heure fixe puis envoyées au serveur NFS cela pourrait engendrer une saturation du réseau d'administration dû à un envoi massif de logs, compressés dans les noeuds de services. Si nous connaissions les performances et caractéristiques du serveur NFS nous aurions pu lui déléguer les logrotate mais dans le cas échéant nous passerons sur ce problème.

Voici donc l'architecture retenue pour la récupération des logs :

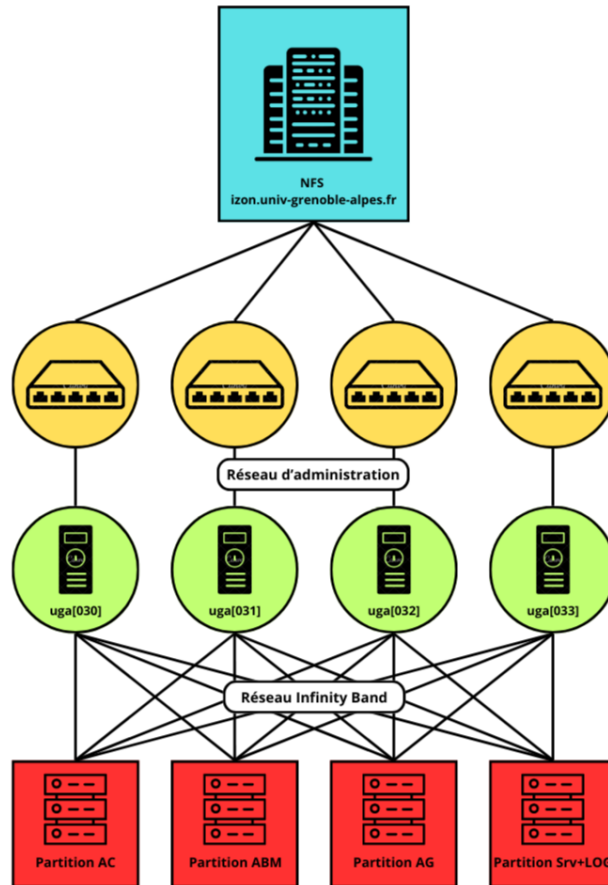


FIGURE 2 – Architecture de la récupération des logs

Il y aura 2 configuration différente : une pour les noeuds de calcul qui enverront les logs aux noeuds de services et une pour les noeuds de calcul qui décideront de les envoyer au serveur en suivant la politique de décision. Il existe 7 niveaux d'urgence pour les logs, 0 étant le plus urgent et 7 le moins urgent. Les noeuds de services enverront seulement les logs évoquant des problèmes au serveur NFS. Voici donc notre politique de gestion des logs du cluster en fonction du niveau d'urgence :

- Code 0 à 2 (resp. emerg, alert, crit) : ces niveaux d'urgence signifient que le système inutilisable, critique nécessitant une intervention immédiate ou critique. Ces logs sont les plus urgents et ont besoin d'être transmit le plus vite possible. Il sont aussi nécessaire si des problèmes récurrents surviennent sur l'infrastructure. C'est pourquoi il seront stocké pendant une durée de 1 an et "rotate" et compressé tous les mois.
- Code 3 (err) : ce niveau d'urgence correspond à un message d'erreur, pas urgent mais nécessitant une attention assez rapidement. Dans cette catégorie seront ajouté les messages lié au LDAP, DNS et au boot. Ils seront conservé 6 mois et "rotate" et compressé tous les mois.
- Code 4 à 7 (res. warning, notice, info, debug) : ce sont les messages les plus nombreux et seront conservés et supprimé au bout de 7 jours. Ce sont les messages les plus

lourds et les moins urgent, c'est pourquoi ils ne seront pas compressé puis "rotate"

Niveau	code	Description
7	debug	Messages de débogage.
6	info	Message d'information.
5	notice	Message d'information un peu importants.
4	warning	Message d'avertissement.
3	err	Message d'erreur.
2	crit	Situation critique.
1	alert	Situation critique nécessitant une intervention immédiate.
0	emerg	Système inutilisable.

TABLE 6 – Niveau d'alerte des logs

Pour ce qui est des console des noeuds, nous pouvons utiliser le logiciel "conman" pour visualiser les logs de plusieurs machines en même temps.

8.2 Fichier de configuration

Voici à présent les différents fichiers de configuration. Il y en a 3. Pour commencer, `rsyslog.service.conf` qui est la configuration sur les noeuds de services :

```

1
2 ### MODULES ###
3 $ModLoad imuxsock # provides support for local system logging (e.g. via
   logger command)
4 $ModLoad imjournal # provides access to the systemd journal
5
6 # Provides TCP syslog reception
7 $ModLoad imtcp
8 $InputTCPServerRun 514
9
10 ### GLOBAL DIRECTIVE ###
11 # Where to place auxiliary files
12 $WorkDirectory /var/lib/rsyslog
13
14 # Use default timestamp format
15 $ActionFileDefaultTemplate RSYSLOG_TraditionalFileFormat
16
17 #Include all config files in /etc/rsyslog.d/
18 $IncludeConfig /etc/rsyslog.d/*.conf
19
20 # Turn off message reception via local log socket;
21 # local messages are retrieved through imjournal now.
```

```
22 $OmitLocalLogging on
23
24 # File to store the position in the journal
25 $IMJournalStateFile imjournal.state
26
27 ### RULES ###
28 # Log anything that is an avenger level threat
29 *.panic /var/cluster/logs/$HOSTNAME%/panic.log
30
31 # Log alert
32 *.alert /var/cluster/logs/$HOSTNAME%/alert.log
33
34 # Log critical
35 *.crit /var/cluster/logs/$HOSTNAME%/critical.log
36
37 # Log anything else
38 *debug /var/cluster/logs/$HOSTNAME%/messages.log
39
40 kern.*
41     /var/cluster/soles/$HOSTNAME%/messages
42
43 ##### JOURNAL #####
44 *.* /var/cluster/logs/%HOSTNAME%/messages
45
46 ### DEFAULT ###
47 # The authpriv file has restricted access.
48 authpriv.* /var/cluster/logs/
49
50 # Log all the mail messages in one place.
51 mail.* -/var/cluster/logs/$HOSTNAME%/maillog
52
53 # Log cron stuff
54 cron.* /var/cluster/logs/$HOSTNAME%/cron
55
56 # Save news errors of level crit and higher in a special file.
57 uucp,news.crit /var/cluster/logs/$HOSTNAME%/spooler
58
59 # Save boot messages also to boot.log
60 local7.* /var/cluster/logs/$HOSTNAME%/boot.log
61
62 # LDAP
63 local4.* /var/cluster/logs/$HOSTNAME%/ldap.log
64
65 ##### CONMAN #####
66 if $programname == 'conmand' then {
67     action(type="omfile" file="/var/cluster/logs/%HOSTNAME%/conman.log")
68 }
```

Listing 1 – rsyslog_service.conf

Et pour finir on a `rsyslog_node.conf` qui est la configuration des noeuds de calcul :

1

```
2 # enable load balancing (https://www.rsyslog.com/doc/configuration/modules
   /omfwd.html)
3 # every 300 seconds, the next target is selected
4 $ActionSendTCPRebindInterval 300
5
6 *.* @@uga030.ib:514
7 ActionExecOnlyWhenPreviousIsSuspended on
8 # si suspend, alors on envoie les logs sur le serveur de secours (failover
   )
9 & @@uga031.ib:514
10 & @@uga032.ib:514
11 & @@uga033.ib:514
12
13 # reset pour le prochain selecteur
14 $ActionExecOnlyWhenPreviousIsSuspended off
```

Listing 2 – rsyslog.node.conf

Et pour finir on a /etc/logrotate.d/rsyslog_rotate.conf qui va gérer les rotations :

```
1 # Compression lors de la rotation
2 compress
3
4 /var/cluster/logs/*/panic.log
5 /var/cluster/logs/*/alert.log
6 /var/cluster/logs/*/crit.log{
7     rotate 12 # 12 mois de retention
8     monthly #rotation mensuelle
9     missingok # pas d'erreur si le fichier est absent
10    notifempty # pas de rotation si le fichier est vide
11    create 0644 root root # crée un nouveau fichier vide après la rotation
   avec les permissions 0644 et la propriété root
12    endscrip
13 }
14
15 /var/cluster/logs/*/error.log {
16     rotate 6 # 6 mois de retention
17     monthly # rotation mensuelle
18     missingok
19     notifempty
20     create 0644 root root
21     endscrip
22 }
23 }
24
25 /var/cluster/logs/*/ldap.log
26 /var/cluster/logs/*/boot.log
27 /var/cluster/logs/*/warning.log {
28     rotate 1 # rotation hebdomadaire
29     weekly
30     missingok
31     notifempty
32     create 0644 root root
```

```
33     endscript
34 }
35
36 /var/cluster/logs/*/messages{
37     rotate 3 # 3 jours d'archives conservées
38     daily
39     missingok
40     notifempty
41     create 0644 root root
42     endscript
43 }
```

Listing 3 – /etc/logrotate.d/rsyslog_rotate.conf

9 Synchronisation temporelle du cluster

On choisit les 3 nœuds de service `uga{030,034,035}`, afin d’implémenter une architecture NTP interne au cluster, qui s’appuie sur l’infrastructure NTP existante de l’université.

Ainsi, chacun de ces 3 nœuds de service hébergent un serveur NTP. Afin de s’appuyer sur l’infrastructure NTP existante de l’université, ces derniers sont synchronisés en mode client/-serveur avec les deux serveurs NTP pré-existant de l’université, en utilisant la connexion du réseau Backbone des nœuds de service. Les deux serveurs NTP de l’université, sont : `charande1.univ-grenoble-alpes.fr` et `charande2.univ-grenoble-alpes.fr`.

Les trois serveurs NTP hébergés par les 3 nœuds de service, sont aussi synchronisés en mode symétrique.

Si les deux serveurs NTP de l’université ne sont plus en état de fonctionner, ou si la connexion au réseau Backbone est coupé, alors les trois serveurs NTP hébergés par les nœuds de service passent en mode symétrique afin de se synchroniser, et de conserver une heure fiable et précise entre eux.

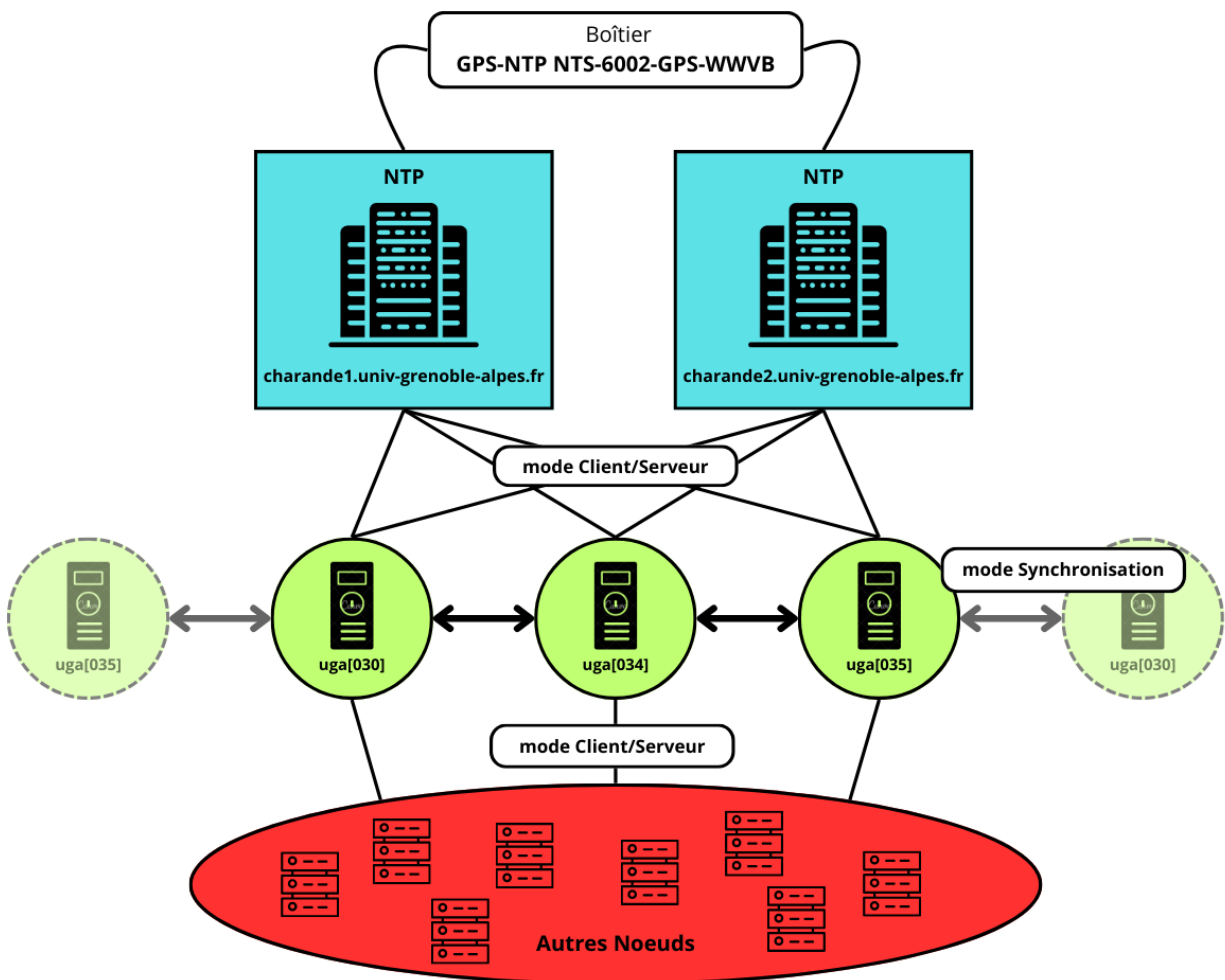


FIGURE 3 – Schéma de l'architecture NTP

Voici les fichiers de configurations :

```

1 # Serveur public de l'université / Mode client/serveur
2 server 172.6.26.15 iburst
3 server 172.6.26.16 iburst
4
5 # Mode symétrique
6 peer uga030.admin (#À commenter pour le serveur NTP sur uga[030])
7 peer uga034.admin (#À commenter pour le serveur NTP sur uga[034])
8 peer uga035.admin (#À commenter pour le serveur NTP sur uga[035])
9
10 # Accès par défaut
11 restrict default nomodify notrap nopeer noquery
12
13 # Horloge de secours, si aucune connexion ne fonctionne (Stratum 3, car le
14   boîtier est à la stratum 1, et les serveurs de l'université à la
15   stratum 2)
16 server 127.127.1.0

```

```
15 fudge 127.127.1.0 stratum 3
```

Listing 4 – Fichier de configuration ntp des serveurs NTP hébergés sur les nœuds de service

```
1 # Mode client/serveur avec les noeud de service
2 server uga030.admin iburst
3 server uga034.admin iburst
4 server uga035.admin iburst
5
6 # Accès par défaut
7 restrict default nomodify notrap nopeer noquery
8
9 # Horloge de secours, si aucune connexion ne fonctionne (Stratum 4, car
   couche en-dessous de la stratum 3)
10 server 127.127.1.0
11 fudge 127.127.1.0 stratum 4
```

Listing 5 – Fichier de configuration ntp pour les autres nœuds (/ les nœuds clients)

Il ne devrait pas être nécessaire de modifier les configurations des serveurs NTP de l'université. En effet, ceux-ci sont publics, et donc les serveurs NTP sur les nœuds de service devraient pouvoir y accéder sans problème.

10 Architecture LDAP

Le cluster s'appuie sur l'infrastructure LDAP déjà en place à l'université, représentée par deux serveurs principaux : la-sure1.univ-grenoble-alpes.fr (172.6.26.11) et la-sure2.univ-grenoble-alpes.fr (172.6.26.12). Ces serveurs contiennent l'annuaire global des utilisateurs (groupes Unix guXX), incluant les comptes nécessaires à l'accès au cluster. Afin de garantir une authentification fluide et sécurisée pour tous les utilisateurs du cluster, tout en assurant une indépendance partielle vis-à-vis de l'extérieur, une infrastructure LDAP interne au cluster est déployée sur trois nœuds de service (uga030, uga031, uga032), configurés en réplication multi-maître.

L'un de ces trois nœuds, uga032, est chargé de se synchroniser régulièrement avec l'un des serveurs LDAP de l'université (par défaut la-sure1) via le protocole syncrepl. Les deux autres nœuds de service (uga030 et uga031) répliquent leur annuaire à partir de uga032, assurant une redondance complète et la tolérance aux pannes. En cas de défaillance de uga032, les serveurs restants peuvent être configurés pour reprendre la synchronisation ascendante. Grâce à cette architecture, les requêtes LDAP internes des nœuds du cluster ne sortent jamais du réseau d'administration, ce qui réduit la latence et améliore la résilience du système.

Les clients LDAP (tous les nœuds du cluster) sont configurés pour interroger les trois serveurs internes. L'annuaire local contient une copie filtrée de l'annuaire universitaire, limitée aux utilisateurs appartenant aux groupes gu[0-9], ainsi qu'à certaines branches nécessaires (ou=users, ou=groups). Ces groupes sont utilisés pour gérer les accès aux ressources partagées du cluster (nœuds de login, stockage, services).

Voici un schéma qui résume notre infrastructure :

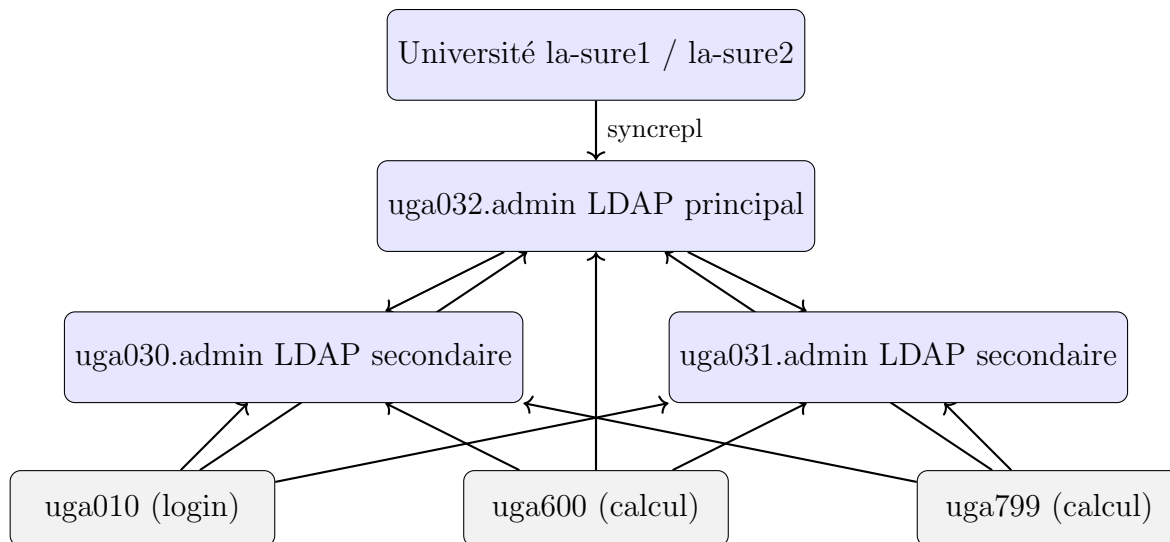


FIGURE 4 – Architecture LDAP interne au cluster

Avec ce que nous venons de dire, nous créons la configuration pour les clients LDAP `/etc/ldap/ldap.conf` :

```
1 BASE dc=cluster,dc=univ-grenoble-alpes,dc=fr
2 URI ldap://uga032.admin ldap://uga030.admin ldap://uga031.admin
3
4 TLS_CACERT /etc/ssl/certs/ca-certificates.crt
5 SASL_MECH gssapi
```

Listing 6 – `/etc/ldap/ldap.conf`

La configuration client LDAP (`/etc/ldap/ldap.conf`) définit la base de recherche dans l'annuaire (`dc=cluster,dc=univ-grenoble-alpes,dc=fr`) et liste plusieurs serveurs LDAP internes à interroger, assurant ainsi redondance et disponibilité. Elle utilise l'authentification sécurisée via Kerberos (SASL/GSSAPI) et des connexions TLS vérifiées par une autorité de certification.

Et du côté serveur LDAP il faut définir la configuration suivante `/etc/openldap/slapd.conf` :

```
1 include /etc/openldap/schema/core.schema
2 include /etc/openldap/schema/cosine.schema
3 moduleload synctprov.la
4
5 ServerID 1 "ldap://uga030.admin"
6 ServerID 2 "ldap://uga031.admin"
7 ServerID 3 "ldap://uga032.admin"
8
9 overlay synctprov
10
11 syncrepl rid=001
12     provider=ldap://la-sure1.univ-grenoble-alpes.fr
13     type=refreshAndPersist
14     interval=00:00:00:10
```

```
15     retry="5 10 60 +"
16     searchbase="dc=univ-grenoble-alpes,dc=fr"
17     filter="(|(gidNumber=10)(gidNumber>=1000))"
18     scope=sub
19 MirrorMode on
20
21 #####
22
23 TLSCertificateFile /etc/ssl/certs/ldap.pem
24 TLSCertificateKeyFile /etc/ssl/private/ldap.key
25
26 database mdb
27 suffix "dc=cluster,dc=univ-grenoble-alpes,dc=fr"
28 directory /var/lib/ldap
29 rootdn "cn=admin,dc=cluster,dc=univ-grenoble-alpes,dc=fr"
```

Listing 7 – /etc/openldap/slapd.conf

La configuration serveur LDAP (`slapd.conf`) met en place une instance multi-maître avec réplication en continu (`sync repl`) à partir du serveur LDAP de l’université. Elle utilise le module `syncprov`, sécurise les échanges avec TLS, et filtre les objets importés pour ne synchroniser que les comptes nécessaires au cluster.

10.1 Changements nécessaires côté LDAP universitaire

11 Architecture DNS

Avant que nous commençons à expliquer notre choix d’architecture pour le service DNS interne au cluster, il est important de définir les conventions de nommage et les plages d’adresses IP à utiliser en prenant en compte la configuration existante.

11.1 Conventions de nommage et adresses IP

Afin de mettre en place une résolution des noms d’hôte à l’intérieur, nous mettons en place un serveur DNS interne dédié. Ce serveur sera utilisé exclusivement depuis le réseau d’administration pour garantir la sécurité et la cohérence des informations de résolution. Afin de faciliter la maintenance et l’extensibilité du cluster, nous adoptons une convention d’adressage systématique basée sur les plages IP suivantes :

Spectre	Plage IP
Administration	172.10.0.0/16
InfiniBand	172.11.0.0/16
Contrôle électrique	172.12.0.0/16
Backbone (interconnexion avec le réseau universitaire)	172.13.0.0/16

TABLE 7 – Plage IP du cluster

Chaque noeud dispose d'une IP unique par réseau, et la correspondance entre adresses et nom d'hôte est déduite de la numérotation logique spécifié dans le tableau 1.

Les noms de domaines internes utilisent des suffixent courts et explicites :

- `ugaXXX.admin` pour l'adresse sur le réseau d'administration
- `ugaXXX.ib` pour l'adresse sur le réseau InfiniBand
- `ugaXXX.ctrl` pour le réseau de contrôle électrique

Les noeuds exposés sur le réseau backbone (noeuds de login, services, maitres, I/O) auront également des noms public du type :

- `hpcX.uga.univ-grenoble-alpes.fr` pour les noeuds de login
- `srvX.uga.univ-grenoble-alpes.fr` pour les noeuds de services
- `ioX.uga.univ-grenoble-alpes.fr` pour les routeurs
- `adminX.uga.univ-grenoble-alpes.fr` pour les noeuds master

Cette convention permet une lecture directe de la topologie du cluster à partir du nom d'hôte, tout en assurant l'unicité des noms. Voici des exmples qui montre l'utilisation de ces convention en cas réel :

Type de machine	Nom / Domaine privé	Admin (172.10.Y.Z)	IB (172.11.Y.Z)	Domaine public (si applicable)
Noeud maître 1	uga020.admin	100.20	100.20	admin0.uga.univ...
Noeud maître 2	uga021.admin	100.21	100.21	admin1.uga.univ...
Service 0	uga030.admin / ib	100.30	100.30	srv0.uga.univ...
Service 1	uga031.admin / ib	100.31	100.31	srv1.uga.univ...
Login 0	uga010.admin / ib	100.10	100.10	hpc0.uga.univ...
Login 1	uga011.admin / ib	100.11	100.11	hpc1.uga.univ...
Calcul AC #0	uga300.admin / ib	103.00	103.00	–
Calcul AG #10	uga600.admin / ib	106.00	106.00	–
Routeur I/O #0	uga050.admin / ib	100.50	100.50	io0.uga.univ...
Serveur NFS	ugaNFS.admin	100.100	–	nfs.uga.univ...

TABLE 8 – Adresses IP et noms DNS internes du cluster)

11.2 Configuration et architecture

Nous choisissons d'utiliser deux noeuds de services qui vont héberger le service DNS interne en mode redondant : uga31 et uga32. Ils sont configurés comme serveurs DNS autoritaires pour les domaines internes (`admin`, `ib`, `ctrl`) et agissent également en tant que caches récursifs pour les noms externes.

Le fonctionnement est le suivant :

- Les zones internes sont définies statiquement dans BIND
- Les requêtes externes sont transmises (forwarding) aux serveurs DNS de l'université (`la-buffe[1-2].univ-grenoble-alpes.fr`)
- Les zones inverses (`in-addr.arpa`) sont également gérés localement pour les IP internes (`172.10.11.12`)
- Les deux serveurs DNS sont indépendants l'un de l'autre pour éviter l'effet maître/esclave, tout en partageant les mêmes fichiers de zones mis-à-jour via Puppet. Ainsi nous évitons les SPOF.
- Les nœuds uga31,32 sont connectés aux switches du backbone (cf. table 4), ce qui leur permet d'échanger avec les serveurs DNS de l'université pour les requêtes externes.

Dans ce fonctionnement les deux serveurs DNS (uga31, uga32) ne sont pas maîtres/esclaves mais configuré de façon identique et autonome et chacun gère l'ensemble des zones internes. Ils sont accessibles via le réseau admin et listé dans la configuration DNS des clients. Enfin ils forwardent les requêtes externes vers l'université mais ne dépendent pas l'un de l'autre. Ainsi une panne de l'un des deux n'empêche pas la résolution DNS du cluster, ce qui est essentiel étant donné l'importance d'un tel service.

Nous définissons donc ces fichiers de configuration pour les serveurs DNS uga31 et uga32 :

```
1 acl cluster {
2     172.10.0.0/16;
3     172.11.0.0/16;
4     172.12.0.0/16;
5 };
6
7 acl univdns {
8     172.6.26.13; // la-buffe1.univ-grenoble-alpes.fr
9     172.6.26.14; // la-buffe2.univ-grenoble-alpes.fr
10 };
11
12 options {
13     listen-on port 53 { 127.0.0.1; 172.10.100.31; }; # IP du serveur
14     directory "/var/named";
15     recursion yes;
16     allow-query { localhost; cluster; };
17     forward only;
18     forwarders { univdns; };
19
20     dnssec-enable yes;
21     dnssec-validation yes;
22     bindkeys-file "/etc/named.root.key";
23     managed-keys-directory "/var/named/dynamic";
24     pid-file "/run/named/named.pid";
25     session-keyfile "/run/named/session.key";
```

```
26 };
27
28 logging {
29     channel logfile {
30         file "/var/log/named.log";
31         severity dynamic;
32         print-category yes;
33         print-time yes;
34     };
35     category queries { logfile; };
36     category config { logfile; };
37 };
38
39 zone "." IN {
40     type hint;
41     file "named.ca";
42 };
43
44 zone "admin" IN {
45     type master;
46     file "named.admin";
47 };
48
49 zone "ib" IN {
50     type master;
51     file "named.ib";
52 };
53
54 zone "ctrl" IN {
55     type master;
56     file "named.ctrl";
57 };
58
59 zone "10.172.in-addr.arpa" IN {
60     type master;
61     file "named.10.172";
62 };
63
64 zone "11.172.in-addr.arpa" IN {
65     type master;
66     file "named.11.172";
67 };
68
69 zone "12.172.in-addr.arpa" IN {
70     type master;
71     file "named.12.172";
72 };
73
74 include "/etc/named.rfc1912.zones";
75 include "/etc/named.root.key";
```

Listing 8 – Configuration de BIND sur le DNS uga31 /etc/named.conf

Le fichier /etc/named.conf est la configuration principale du serveur BIND sur le DNS

uga31

```

1
2 @      IN      SOA  ns1.admin. root.admin. (
3        2025042501
4        3H
5        1H
6        3W
7        1D      )
8 @      IN      NS   ns1.admin.
9 ns1    IN      A     172.10.100.31
10 uga020 IN      A     172.10.100.20

```

Listing 9 – /var/named/named.admin,named.in,named.ctrl

Les fichiers `named.admin`, `named.in`, `named.ctrl` fonctionnent tous les trois de la même manière et décrivent la correspondance nom de domaine - adresse IP dans les domaines internes. Dans cet exemple nous explicitons la correspondance entre le serveur DNS1 appelé `ns2` à l'IP de notre noeud de service, hébergeant le DNS, `172.10.100.31`.

```

1
2 $TTL 86400
3 @      IN      SOA  ns1.admin. root.admin. (
4        2025042501 ; Serial
5        3600       ; Refresh
6        1800       ; Retry
7        604800     ; Expire
8        86400      ; Minimum TTL
9
10 @      IN      NS   ns1.admin.
11
12 20.100 IN      PTR  uga020.admin.
13 21.100 IN      PTR  uga021.admin.
14 30.100 IN      PTR  uga030.admin.
15 31.100 IN      PTR  uga031.admin.
16 100.100 IN     PTR  ugaNFS.admin.

```

Listing 10 – /var/named/named.10.172,named.11.172

Ces fichiers permettent la résolution inverse (adresse IP en nom de domaine). Ils sont utiles pour des logs, SSH, ou certains services qui veulent connaître le FQDN d'un client. Chacun des fichiers `named.10.172` (admin), `named.11.172` (IB), `named.12.172` (contrôle) gèrent une plage réseau.

De l'autre côté les client ont la configuration suivante pour `/etc/resolv.conf`

```

1 search admin ib ctrl uga.univ-grenoble.fr
2 nameserver 172.10.100.30 # uga030
3 nameserver 172.10.100.31 # uga031

```

Listing 11 – /etc/resolv.conf des clients

Ce paramétrage garanti que les résolutions locales utilisent d'abord les DNS internes et que si l'un est indisponible, le second est utilisé. Il faut aussi modifier `/etc/nsswitch.conf`

qui indique dans quel ordre le système va chercher les informations liées aux services réseau. Ainsi avec la configuration ci-dessous, pour résoudre un nom de machine, le système regarde d'abord `/etc/hosts` puis fait une requête DNS si la première ne trouve aucune correspondance.

```
1 hosts: files dns
```

Listing 12 – `/etc/nsswitch.conf` des clients

11.3 Changements requis côté DNS de l'université

Afin d'assurer l'intégration du cluster dans l'infrastructure réseau existante de l'université, certaines machines doivent être rendues accessibles depuis l'extérieur, notamment via le réseau backbone universitaire. Ces machines — principalement les nœuds de login, les nœuds maîtres, les nœuds de service et les routeurs I/O — nécessitent donc une présence dans le DNS de l'université. Bien que le cluster dispose de son propre DNS interne pour la résolution locale (avec des zones privées telles que `admin`, `ib`, `ctrl`), celui-ci n'est pas exposé publiquement. Pour que les utilisateurs puissent se connecter à un nœud de login, ou pour que les administrateurs puissent accéder aux nœuds maîtres, il est indispensable que ces machines soient enregistrées dans la zone DNS publique de l'université, typiquement `uga.univ-grenoble.fr`.

Le nommage public adopté suit une convention simple et explicite, distincte du nommage interne afin de limiter la fuite d'informations techniques tout en facilitant l'identification des rôles de chaque machine. Par exemple, le nœud de login `uga010.admin` sur le réseau d'administration se voit attribuer un enregistrement DNS public sous le nom `hpc0.uga.univ-grenoble.fr`, tandis que le nœud maître `uga020.admin` devient `admin0.uga.univ-grenoble.fr`. Ces noms sont associés à leurs adresses IP respectives sur le réseau backbone (`172.13.0.0/16`), comme indiqué dans le tableau suivant :

Machine	Adresse IP Backbone	Nom DNS public
Nœud de login 0 (uga010)	172.13.0.10	<code>hpc0.uga.univ-grenoble.fr</code>
Nœud de login 1 (uga011)	172.13.0.11	<code>hpc1.uga.univ-grenoble.fr</code>
Nœud maître 0 (uga020)	172.13.0.20	<code>admin0.uga.univ-grenoble.fr</code>
Nœud maître 1 (uga021)	172.13.0.21	<code>admin1.uga.univ-grenoble.fr</code>
Service DNS (uga031)	172.13.0.30	<code>srv0.uga.univ-grenoble.fr</code>
Service secondaire (uga032)	172.13.0.31	<code>srv1.uga.univ-grenoble.fr</code>
Routeur I/O 0 (uga050)	172.13.0.50	<code>io0.uga.univ-grenoble.fr</code>

TABLE 9 – Enregistrements DNS à ajouter côté université

Ces enregistrements de type A devront être intégrés dans la zone DNS de l'université par le service réseau, de manière à permettre la résolution de ces noms depuis n'importe quel réseau universitaire (et éventuellement depuis l'extérieur, sous conditions de filtrage). Ce découplage clair entre le DNS interne (privé) et le DNS externe (public) renforce la sécurité et permet une gestion autonome du cluster tout en maintenant l'interopérabilité avec l'infrastructure existante.

12 Configuration du contrôleur SLURM

12.1 Choix des serveurs et installation des dépendances

Afin de pouvoir utiliser SLURM, on doit d'abord installer les dépendances nécessaires sur ces noeuds de calculs ! À savoir, **munge** et **slurmd**.

En effet, tout les démons slurms ont besoin de **munge** pour fonctionner correctement, c'est pourquoi il est nécessaire de l'installer sur tout les noeuds utilisés par Slurm ! C'est le cas des noeuds de calculs, ceux-ci ont besoin du démon slurmd afin de pouvoir exécuter les jobs qui leur sont donnés correctement !

Ainsi, pour les noeuds qui doivent gérer l'organisation de Slurm, il en va de soit qu'on devra aussi installer **munge** sur ces derniers ! Le démon **slurmctld** sera aussi nécessaire afin de gérer et ordonner le cluster avec Slurm, ce qui vaut de même pour le démon **slurmdbd**, utilisé afin de gérer le stockage des informations sur les jobs ! Enfin, on utilisera **MariaDB** afin de stocker les informations obtenus par le démon **slurmdbd**.

En ce qui concerne les noeuds qui seront utilisés, nous en utiliserons deux noeuds de service : **uga[034]** et **uga[035]** ! Nous avons préféré utilisés des noeuds de services de notre cluster, plutôt que les noeuds maître du cluster afin d'éviter de les surcharger ! D'autant plus, que le rôle des noeuds maître est primordiale pour le bon fonctionnement du cluster ! De plus, l'utilisation des noeuds de service pour gérer Slurm est possible, donc on a choisi de partir vers cette direction.

Ainsi, ces deux noeuds de service agiront comme des noeuds "maître" pour Slurm ! L'utilisation de deux noeuds, et non d'un réside dans une volonté d'éviter un SPOF. En effet, un noeud (**uga[034]**) sera utilisé comme noeud "maître" principal de Slurm, tandis qu'un deuxième (**uga[035]**) sera utilisé comme noeud "maître" de secours dans cas où un problème intervient avec le premier !

Ces noeuds seront utilisés à travers leur connexion au réseau d'administration ! Car leur rôle sera de gérer le cluster à travers Slurm !

12.2 Configuration Slurm

```
1 ClusterName=uga
2 SlurmctldHost=uga034.admin
3 SlurmctldHost=uga035.admin
4 AuthType=auth/munge
5 #
6 #DisableRootJobs=NO
7 #EnforcePartLimits=NO
8 #Epilog=
9 #EpilogSlurmctld=
10 #FirstJobId=1
11 #MaxJobId=67043328
12 #GresTypes=
13 #GroupUpdateForce=0
14 #GroupUpdateTime=600
15 #JobFileAppend=0
16 #JobRequeue=1
17 #JobSubmitPlugins=lua
18 #KillOnBadExit=0
19 #LaunchType=launch/slurm
20 #Licenses=foo*4,bar
21 #MailProg=/bin/mail
22 MaxJobsPerUser=100 #Limitation de job par défaut (voir 12.3)
23 MaxSubmitJobsPerUser=300 #Limitation de job par défaut (Voir 12.3)
24 MaxJobCount=10000
25 #MaxStepCount=40000
26 #MaxTasksPerNode=512
27 MpiDefault=none
28 #MpiParams=ports=-#
29 #PluginDir=
30 #PlugStackConfig=
31 #PrivateData=jobs
32 ProctrackType=proctrack/cgroup
33 #Prolog=
34 #PrologFlags=
35 #PrologSlurmctld=
36 #PropagatePrioProcess=0
37 #PropagateResourceLimits=
38 #PropagateResourceLimitsExcept=
39 #RebootProgram=
40 ReturnToService=1
41 SlurmctldPidFile=/var/run/slurmctld.pid
42 SlurmctldPort=6817
43 SlurmdPidFile=/var/run/slurmd.pid
44 SlurmdPort=6818
45 SlurmdSpoolDir=/var/spool/slurmd
46 SlurmUser=root
47 #SlurmdUser=root
48 #SrunEpilog=
49 #SrunProlog=
50 StateSaveLocation=/var/spool/slurmctld
51 SwitchType=switch/none
```

```
52 #TaskEpilog=
53 TaskPlugin=task/affinity,task/cgroup
54 #TaskProlog=
55 TopologyPlugin=topology/tree
56 #TmpFS=/tmp
57 #TrackWCKey=no
58 #TreeWidth=
59 #UnkillableStepProgram=
60 #UsePAM=0
61 #
62 #
63 # TIMERS
64 #BatchStartTimeout=10
65 #CompleteWait=0
66 #EpilogMsgTime=2000
67 #GetEnvTimeout=2
68 #HealthCheckInterval=0
69 #HealthCheckProgram=
70 InactiveLimit=0
71 KillWait=30
72 MessageTimeout=90
73 #ResvOverRun=0
74 MinJobAge=300
75 #OverTimeLimit=0
76 SlurmctldTimeout=120
77 SlurmdTimeout=300
78 #UnkillableStepTimeout=60
79 #VSizeFactor=0
80 Waittime=0
81 #
82 #
83 # SCHEDULING
84 #DefMemPerCPU=0
85 #MaxMemPerCPU=0
86 #SchedulerTimeSlice=30
87 SchedulerType=sched/backfill
88 SelectType=select/cons_tres
89 SelectTypeParameters=CR_Core
90 #
91 #
92 # JOB PRIORITY
93 #PriorityFlags=
94 PriorityType=priority/multifactor
95 #PriorityDecayHalfLife=
96 #PriorityCalcPeriod=
97 #PriorityFavorSmall=
98 #PriorityMaxAge=
99 #PriorityUsageResetPeriod=
100 #PriorityWeightAge=
101 PriorityWeightFairshare=100000
102 PriorityWeightJobSize=0
103 PriorityWeightPartition=0
104 PriorityWeightQOS=1000000
105 #
```

```

106 #
107 # LOGGING AND ACCOUNTING
108 AccountingStorageEnforce=qos,limits
109 AccountingStorageHost=uga034.admin #On peut aussi utiliser un 3ième noeud
    de service, pour éviter la surcharge sur le m me noeud
110 #AccountingStoragePass=
111 #AccountingStoragePort=
112 AccountingStorageType=accounting_storage/slurmdbd
113 #AccountingStorageUser=
114 #AccountingStoreFlags=
115 #JobCompHost=
116 #JobCompLoc=
117 #JobCompPass=
118 #JobCompPort=
119 JobCompType=jobcomp/mysql
120 #JobCompUser=
121 #JobContainerType=
122 JobAcctGatherFrequency=30
123 JobAcctGatherType=jobacct_gather/cgroup
124 SlurmctldDebug=info
125 SlurmctldLogFile=/var/log/slurmctld.log
126 SlurmdDebug=info
127 SlurmdLogFile=/var/log/slurmd.log
128 #SlurmSchedLogFile=
129 #SlurmSchedLogLevel=
130 #DebugFlags=
131 #
132 #
133 # POWER SAVE SUPPORT FOR IDLE NODES (optional)
134 #SuspendProgram=
135 #ResumeProgram=
136 SuspendTimeout=60
137 ResumeTimeout=300
138 ResumeRate=5
139 #SuspendExcNodes=
140 #SuspendExcParts=
141 SuspendRate=5
142 SuspendTime=600
143 #
144 #
145 # COMPUTE NODES
146 NodeName=uga[100-149] State=UNKNOWN #Partition AMB
147 NodeName=uga[200-249] State=UNKNOWN #Partition AG
148 NodeName=uga[300-601] State=UNKNOWN #Partition AC
149 NodeName=linux[1-32] CPUs=1 State=UNKNOWN
150 PartitionName=AMB Nodes=uga[100-149] Default=YES MaxTime=INFINITE State=UP
151 PartitionName=AG Nodes=uga[200-249] Default=YES MaxTime=INFINITE State=UP
152 PartitionName=AC Nodes=uga[300-601] Default=YES MaxTime=INFINITE State=UP

```

Listing 13 – Fichier de configuration slurm.conf

```

1 # Archive info
2 #ArchiveJobs=yes
3 #ArchiveDir="/tmp"

```

```
4 #ArchiveSteps=yes
5 #ArchiveScript=
6 #JobPurge=12
7 #StepPurge=1
8 #
9 # Authentication info
10 AuthType=auth/munge
11 #AuthInfo=/var/run/munge/munge.socket.2
12 #
13 # slurmDBD info
14 DbdAddr=localhost
15 DbdHost=localhost
16 #DbdPort=7031
17 SlurmUser=root
18 #MessageTimeout=300
19 DebugLevel=verbose
20 #DefaultQOS=normal,standby
21 LogFile=/var/log/slurm/slurmdbd.log
22 PidFile=/var/run/slurmdbd.pid
23 #PluginDir=/usr/lib/slurm
24 #PrivateData=accounts,users,usage,jobs
25 #TrackWCKey=yes
26 #
27 # Database info
28 StorageType=accounting_storage/mysql
29 #StorageHost=localhost
30 #StoragePort=1234
31 StoragePass=<password>
32 StorageUser=root
33 #StorageLoc=slurm_acct_db
```

Listing 14 – Fichier de configuration slurmdbd.conf

```
1 CgroupAutomount=yes
2 CgroupMountpoint=/sys/fs/cgroup
3
4 CgroupReleaseAgentDir="/etc/slurm/cgroup"
5
6 ConstrainCores=yes
7 ConstrainDevices=yes
8 ConstrainRAMSpace=yes
9 TaskAffinity=yes
```

Listing 15 – Fichier de configuration cgroup.conf

Pour le fichier de configuration `topology.conf`, on le génère en utilisant l'outil `slurmibtopology` renseigné sur le site de Slurm. En effet, cet outil permet de créer un fichier `topology.conf` pour obtenir le noeud correct, avec le bon switch Infiniband.

Avec la mise en place de **slurmdbd**, on doit aussi mettre en place **MariaDB**, ou plutôt la base de donnée pour Slurm ! Pour cela, on lance **mysql** en mode root, et on rentre :

```
1 create database slurm_acct_db;
```

```
2 grant all on slurm_acct_db.* TO 'root'@'localhost' identified by '<
    password>' with grant option;
```

Listing 16 – Création de la base de donnée

12.3 Restriction de l'allocation des ressources

Avant de commencer, on doit créer le cluster :

```
1 sacctmgr -i create cluster uga
```

Listing 17 – Création du cluster **uga**

Afin de restreindre les ressources entre les projets, on va créer une partition par projet ! Voici comment seront réparties les projets par rapport au groupe d'utilisateur :

- **projet0** : **gu[0-2]** (30 personnes max)
- **projet1** : **gu[3-4]** (60 personnes max)
- **projet2** : **gu[5-7]** (10 personnes max)
- **projet3** : **gu[8-9]** (40 personnes max)

Afin de limiter le nombre de jobs utilisées par défaut par l'utilisateur, on a rajouté dans le fichier de configuration `slurm.conf` une limite sur le nombre de job maximum qu'un utilisateur peut exécuter en même temps (`MaxJobsPerUser=100`) à 100, et une limite sur le nombre de jobs qu'il peut mettre en attente à 200 (`MaxSubmitJobsPerUser=300`, où 300 représente le nombre de jobs soumis, donc ceux en attente et ceux en exécution).

Pour donner la possibilité aux utilisateurs des projets 1 et 2 de faire tourner rapidement des jobs pour débbugger, tout en restreignant encore plus le nombre de ces jobs et leur durée, on va créer une QOS "debug" avec une priorité élevée, en y incluant les restrictions :

```
1 sacctmgr -i create qos name=debug Priority=10 MaxJobs=5 MaxWall=5
```

Listing 18 – Création de la QOS debug pour les projets 1 et 2

Maintenant, on va créer les partitions de chacun des 4 projets, avec leur pourcentage d'utilisation de la machine autorisé respectif. On va aussi, mettre en place la restriction sur le projet 0, qui l'empêche d'accéder à la partition AmdGpu (AG), en même temps que la création de sa partition. Et enfin, nous préciserons aussi les QOS que ces partitions peuvent utiliser.

```
1 sacctmgr -i create account name=projet0 cluster=uga fairshare=10 partition
    =ABM,AC qos=normal
2 sacctmgr -i create account name=projet1 cluster=uga fairshare=30 partition
    =ABM,AC,AG qos=normal,debug
3 sacctmgr -i create account name=projet2 cluster=uga fairshare=35 partition
    =ABM,AC,AG qos=normal,debug
4 sacctmgr -i create account name=projet3 cluster=uga fairshare=25 partition
    =ABM,AC,AG qos=normal
```

Listing 19 – Création des partitions

Maintenant qu'on a effectué cela, on peut créer les utilisateurs. On va en créer au moins 1 par projet :

```

1 sacctmgr -i add user name=user17gu1projet0 account=projet0
2 sacctmgr -i add user name=user46gu4projet1 account=projet1
3 sacctmgr -i add user name=user9gu5projet2 account=projet2
4 sacctmgr -i add user name=user30gu8projet3 account=projet3

```

Listing 20 – Rajout d’au moins 1 utilisateur par partition/groupe

12.4 Vérification de la politique mise en place

Afin de vérifier si les QOS qu’on a mis en place sont bien paramétrés, on peut effectuer la commande :

```
1 sacctmgr show qos
```

Et pour vérifier les partitions/groupes, et les noeuds on peut effectuer la commande :

```

1 sinfo
2 #ou pour plus d'information
3 sinfo -l

```

Sinon, pour tout vérifier d’un coup, on peut effectuer la commande :

```
1 sacctmgr show associations -P where account=projet0,projet1,projet2,
   projet3 format=Account,User,Fairshare,Partition,QOS
```

13 Configuration de la surveillance

La société NextClu fournit trois sondes principales permettant de vérifier les critères de disponibilité du contrat :

- ”check-node” : vérifie qu’un pourcentage de noeuds est ”up”
- ”check-lustre” : vérifie que Lustre est opérationnel et mesure le débit
- ”check-ssh” : vérifie que le service SSH est disponible sur une liste de noeuds

Voici comment ces sondes peuvent être utilisées pour vérifier les cinq critères du contrat de disponibilité.

1. Disponibilité des noeuds de login, au moins un noeud opérationnel pour chaque communauté

```

1 check-ssh -H uga[000-001] -w 1 -c 0      # communauté gu[0-2]
2 check-ssh -H uga[002-005] -w 1 -c 0      # communauté gu[3-4]
3 check-ssh -H uga[006]      -w 1 -c 0      # communauté gu[5-7]
4 check-ssh -H uga[007-009] -w 1 -c 0      # communauté gu[8-9]

```

2. Débit minimal de 32 Gb/s vers Lustre

```
1 check-lustre -H @io -w 35 -c 31          # seuil de sécurité à 35 GB/s
```

3. Disponibilité de 90% des noeuds de la partition AC

```
1 check-node -H @amd_compute -w 95 -c 89
```

4. Disponibilité de 80% des noeuds de la partition ABM

```
1 check-node -H @amd_big_mem -w 85 -c 79
```

5. Disponibilité de 60% des noeuds de la partition AG

```
1 check-node -H @gpu_compute -w 65 -c 59
```

Nous voulons implémenter un sonde qui surveillera la disponibilité pour une partition SLURM. Pour surveiller dynamiquement le pourcentage de nœuds disponibles dans une partition Slurm, une sonde bash peut être mise en place. Elle se connecte en SSH à un nœud du cluster, exécute `sinfo`, extrait les statistiques de la partition, puis compare le pourcentage de nœuds disponibles à des seuils critiques. Il faudrait ensuite l'utiliser comme ceci :

```
1 check-slurm-partition --partition=ac -H master1 --user=shinken -w 90 -c 85
```

13.1 Analyse des noeuds de login

Pour assurer un service interactif de qualité, il est aussi essentiel de surveiller trois éléments sur les noeuds de login.

Dans un premier temps, il est essentiel de surveiller la charge matérielle (RAM, CPU et stockage) pour gérer les charges de travail des utilisateurs de manière efficace sur un nœud. Il est donc primordial de surveiller son taux de saturation, et c'est en le vérifiant que l'on pourra assurer un service interactif confortable pour les utilisateurs.

Ensuite il y a la disponibilité réseau. Trop de latence dégradera le point de vue utilisateur et nuira à l'expérience car elle entraînera un manque d'interactivité. Il est donc nécessaire de vérifier la latence des nœuds à chaque instant. Elle peut venir de nombreux facteurs, problèmes de certificats, saturation réseau. Si on laissait de la latence, la qualité du service baisserait et on perdrait des clients.

C'est un point évident mais bon à rappeler mais il faut que l'utilisateur puisse se connecter aux nœuds. Et donc son accessibilité est essentielle à surveiller.

Au niveau du cluster, les services critiques essentiels à surveiller sont les nœuds d'administration et les nœuds de services. Les nœuds d'administration, dans un premier temps, sont essentiels pour régler les problèmes dans le cluster, gérer les différentes configurations et faire des maintenances. Quant aux nœuds de services, ils hébergent des services comme le DNS, le LDAP, la récupération des logs. Si les nœuds de services sont HS cela pourrait rendre le cluster inaccessible (plus de DNS et/ou plus de LDAP), rendrait aveugle l'administrateur réseau (plus de gestion des logs). Nous pouvons aussi citer le serveur NFS qui est très important car regroupant un grand nombre de données. La perte de ces dernières porterait un grand préjudice aux clients, tout comme à notre infrastructure.

Pour effectuer une analyse plus fine des performances du cluster, il faudrait mettre en place des checks plus précis avec plus d'options. Nous pourrions aussi utiliser Prometheus avec un outil de visualisation comme Grafana qui pourraient faire ressortir plus facilement les performances, les avantages ou bien les faiblesses de notre cluster. De plus, nous pourrions plus

facilement distribuer ces données à d'autres équipes, comme des data analyst, qui pourrait relever des informations encore plus pertinentes.

14 Configuration Puppet

Afin d'assurer une configuration automatique des nœuds et serveurs du cluster en un temps raisonnable, nous déployons une architecture Puppet basée sur deux serveurs. Le premier serveur Puppet est installé sur un nœud maître, et le second sur un nœud de service. Les deux sont configurés de manière identique et agissent comme serveurs Puppet (puppetserver). Ce choix permet de répondre à deux contraintes majeures : la tolérance aux pannes (redondance, pas de SPOF) et l'accessibilité. Si les nœuds de service deviennent inaccessibles, les nœuds maîtres — qui servent de point de contrôle aux administrateurs — gardent la possibilité de gérer la configuration des autres machines. Cette architecture offre donc à la fois résilience et flexibilité, et si besoin, elle pourra être étendue en ajoutant de nouveaux serveurs Puppet sur d'autres nœuds critiques.

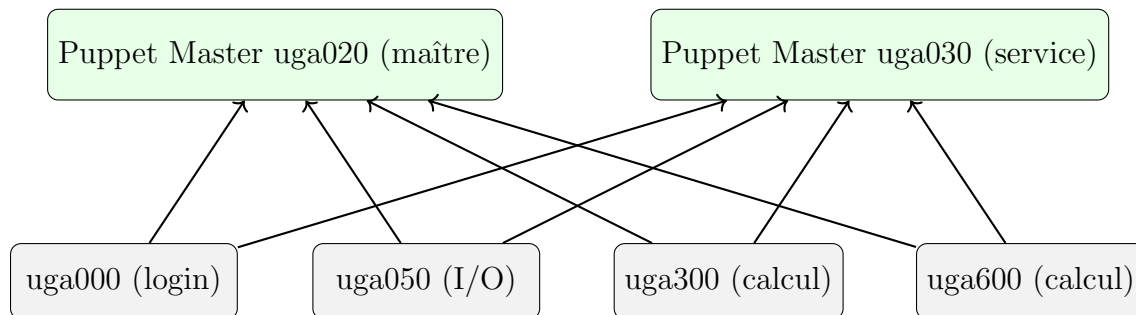


FIGURE 5 – Architecture Puppet avec serveurs redondants et agents

Avant le déploiement de Puppet, l'installation des serveurs DNS internes est obligatoire, car Puppet repose sur une résolution correcte des noms. Les nœuds du cluster doivent tous disposer du paquet puppet-agent, et les deux machines dédiées au service doivent également installer puppetserver. Une fois le service actif, chaque nœud agent doit initier une demande de certificat auprès des serveurs Puppet, qui la signeront pour établir une connexion sécurisée. Ensuite, les configurations seront gérées via des manifests, contenant des classes et des profils qui regroupent les paramètres applicables à chaque rôle du cluster. L'organisation des nœuds se fait par groupes homogènes selon leur usage, et chaque groupe est associé à un ou plusieurs profils Puppet.

Groupe / Rôle	Profils Puppet	Nodeset	Commentaire
Nœuds de calcul	slurm : :client, dns : :client, ib : :config, base : :utils, ib : :hdr	uga[300-600]	Nœuds CPU ou GPU, partition AC, ABM et AG
Nœuds de login	user : :interface, dns : :client, ssh : :server, slurm : :client, ib : :hdr, backbone : :connection	uga[000-009]	Interface utilisateur, accès distant, connexion Backbone
Nœuds maîtres	puppet : :server, dns : :master, ldap : :master, slurm : :control, admin : :tools, shinken : :monitor, dhcp : :master, pxe : :server, ib : :hdr, backbone : :connections	uga[020-021]	Point de contrôle, services critiques, failover DNS et Puppet, connexion Backbone
Nœuds de service	dns : :server, puppet : :server, rsyslog : :server, nfs : :server, dhcp : :secondary, ldap : :slave, ntp : :server, slurm : :database, ib : :hdr, backbone : :connection	uga[030-035]	Haute disponibilité, centralisation des services système, connexion Backbone
Nœuds I/O	lustre : :router, network : :gateway, dns : :client, ib : :hdr, ib : :storage, backbone : :flow	uga[050-055]	Pont entre calcul et stockage Lustre, IB HDR double connexion (calculateur et stockage)
Serveur NFS	nfs : :server, rsyslog : :client, dns : :client	ugaNFS	Stockage partagé (logs, /home, /data)

TABLE 10 – Répartition des rôles, profils et machines dans l’architecture Puppet

L’ensemble des modules nécessaires est en grande partie disponible sur la forge Puppet. Par exemple, les modules pour ldap, ntp, rsyslog, slurm, dhcp, ou encore pxe sont directement exploitables. Toutefois, certains profils devront être développés en interne, notamment ceux liés à la configuration spécifique du cluster (ex. : profile : :lustre : :router pour la gestion des I/O via Lustre, ou profile : :dns : :server avec zones admin, ib, ctrl). Les profils Puppet seront organisés dans des modules hiérarchisés selon leur fonctionnalité, et les variables spécifiques à chaque rôle ou machine seront centralisées via Hiera, avec une arborescence de données du type :

```
hiera/
├── common.yaml
├── roles/
│   ├── compute.yaml
│   ├── login.yaml
│   └── master.yaml
└── nodes/
    ├── uga020.yaml
    └── uga030.yaml
```

Cette séparation entre le code (manifests, modules) et les données (paramètres via Hiera) garantit la maintenabilité de la configuration, tout en assurant une adaptation fine aux besoins du cluster. Elle permet aussi à plusieurs administrateurs de collaborer efficacement, grâce au versionnement de la configuration dans un dépôt Git.

15 Environnement Utilisateur

Pour installer la version la plus récente de CheMPS2 (version 1.8.12), on utilisera la toolchain `foss/2023a` (ou `foss/2022a` `foss/2022b`). Et pour installer la version la plus récente de OpenBLAS (version 0.3.27), on va avoir besoin de la toolchain `GCC/13.3.0` (ou de `GCC/13.2.0`). De plus, pour CheMPS2, et OpenBLAS, on aura aussi besoin de la toolchain `system`, car cette toolchain est nécessaire à GCC et foss, quel que soit la version.

Ainsi, pour l'instant on aura déjà eu besoin des toolchains `system`, `foss/2023a` et `GCC/13.3.0` pour l'installation des versions récentes de CheMPS2 et OpenBLAS.

Pour un environnement de développement GNU de base, on va avoir besoin, en supplément, des toolchains `autotools/20231222`, `make/4.4.1` et `binutils/2.42`. Avec la toolchain `GCC/13.3.0` déjà installé, on devrait avoir désormais un environnement de développement GNU de base.

Afin d'avoir un environnement de développement Intel complet, on va devoir installer la toolchain `intel/2024a` qui fournit à peu près tout ce qui est nécessaire.

Enfin, pour avoir un environnement de développement pour des codes accélérés par GPU/NVIDIA, on installera les toolchains `CUDA/12.8.0` (qui contient un toolkit adapté pour les codes accélérés par GPU) et aussi `CUDAcompat/11` (afin de minimiser les risques de conflit de CUDA avec les autres toolchains, dû à sa version).

On peut installer ces produits dans un système de fichier Lustre. En effet, pour un clus-

ter imposant, tel qu'on fait face avec cette appel d'offre, un système de fichier Lustre est intéressant, car elle offre un accès centralisé et scalable, mais aussi de hautes performances I/O parallèle ! Cependant, ce système de fichier présente quelques inconvénients, sa configuration et son installation sont assez compliqués, et long, de plus ce système de fichier n'est pas très efficace pour les accès fréquents à de nombreux petits fichiers.

16 Déploiement

LTS est un acronyme qui signifie "Long Term Support" ("Support à long terme"). Une LTS a pour caractéristique d'être maintenue avec des mises à jour régulière pendant plusieurs années en générale. Une LTS garantit aussi une forme de stabilité, et encore plus que cela, ses mises à jour ont tendances à privilégier sa stabilité et sa sécurité plutôt qu'autre chose. Ainsi, il arrive qu'une LTS ne soit pas au top de la nouveauté, car ce n'est pas l'objectif principale de ses mises à jour.

RHEL 9.3 est supporté par la version MOFED -2310213 LTS. Les cartes ConnectX-4, ConnectX-4 Lx, ConnectX-5, ConnectX-5 Ex, ConnectX-6, ConnectX-6 Dx, ConnectX-6 Lx et ConnectX-7 sont supportés par cette version de MOFED. Cependant, on a besoin d'utiliser un firmware qui varie pour chacune de ces cartes ConnectX, pour qu'une carte puisse vraiment être compatible avec MOFED. La version de firmware qu'on doit utiliser avec la carte ConntectX-6 est la version 20.39.3004. Cette version des drivers Infiniband supportent les versions de Lustre 2.15.2 et 2.12.9.

C'est la version de Lustre 2.15.4 qui est compatible avec la version RHEL 9.3, selon la page.

RedHat	Lustre	MOFED	Firmware ConnectX-6
RHEL 9.3	Lustre 2.15.4 ¹	MOFED -2310213 LTS	20.39.3004

TABLE 11 – Matrice de compatibilité

Une fois les noeuds maîtres installés,

17 Planification de l'installation du cluster

Afin de permettre à l'université d'utiliser le cluster et de le mettre au service de ses étudiants, il est essentiel que tous les services soient opérationnels le plus rapidement possible. Il y a deux éléments qui sont réellement indispensables pour démarrer l'installation :

- Les racks des switch Ethernet et InfiniBand (rkwl)
- Les 2 racks de service (rkSrv1 et rkSrv2) contenant les noeuds maîtres, services, login, routeurs, NFS

Ces éléments sont essentiels pour configurer le réseau d'administration, déployer l'infrastructure logique (SLURM, Puppet, rsyslog etc) et monter les partitions de calcul dès leur livraison.

Les étapes de mise en production du cluster sont donc les suivantes :

1. Livraison et installation physique des racks
2. Câblage (alimentation, réseau, interconnexion)
3. Intégration logicielle des nœuds et tests d'infrastructure
4. Installation des services systèmes et logiciels (SLURM, LDAP, DNS, Puppet, Lustre ...)
5. Vérification d'Aptitude au Bon Fonctionnement (VABF) en environnement de test (tiré de cette source croisé avec d'autres sources sur internet))
6. Vérification de Service Régulier (VSR) en environnement réel (information tirée de ce site croisé avec d'autres sources sur internet)
7. Correction éventuelles
8. Mise en production progressive par partition et bout de partition

Pour l'élaboration du diagramme de Gantt, nous devons estimer la durée de ces étapes. Nous n'avons pas trouvé de données réellement fiables ou pertinentes qui s'adaptent à notre cas, quand à la durée de la VABF et de la VSR. Nous estimerons cette durée à 1 semaine pour les deux car c'est un chiffre qui ressort souvent quand on cherche sur internet. La VABF doit être effectuée pour chaque rack et est suivie par la VSR. Pour finir il y a 15 jours de corrections et de rectifications, très importants avant la mise en production. À la fin de chacune de ces étapes (VABF, VSR, correction), nous pensons que la société doit remplir de la paperasse (des certifications, des compte-rendus) sur une durée que nous mettons à 1 jour.

Pour remettre dans le contexte, la société NextClu ne pourra livrer que 2 racks par semaine à partir de la semaine 18/2025, uniquement le mercredi matin à 8h00. Chaque rack nécessite 1 journée complète pour être installé (matin : installation physique, après-midi : intégration et raccordement). Aucun travail n'est effectué les week-ends ni jours fériés, sauf les VABF et VSR qui sont des tests en environnement contrôlé et en production qui peuvent être laissés en train de tourner le week-end. Voici donc un tableau qui résume les dates de livraisons :

Semaine	Racks	Raison
18	rkW1, rkSrv1	Réseau + nœuds critiques
19	rkSrv2, rkAC1	Reste des services + début partition de calcul
20	rkAC2, rkAC3	Suite partition AC
21	rkAC4, rkABM1	Fin AC + ABM complet
22	rkAG1, rkAG2	Début AG
23	rkAG3, réserve	Fin AG

TABLE 12 – Ordre de livraison des racks du cluster

Nous pouvons donc éditer le diagramme de Gantt de la commande :

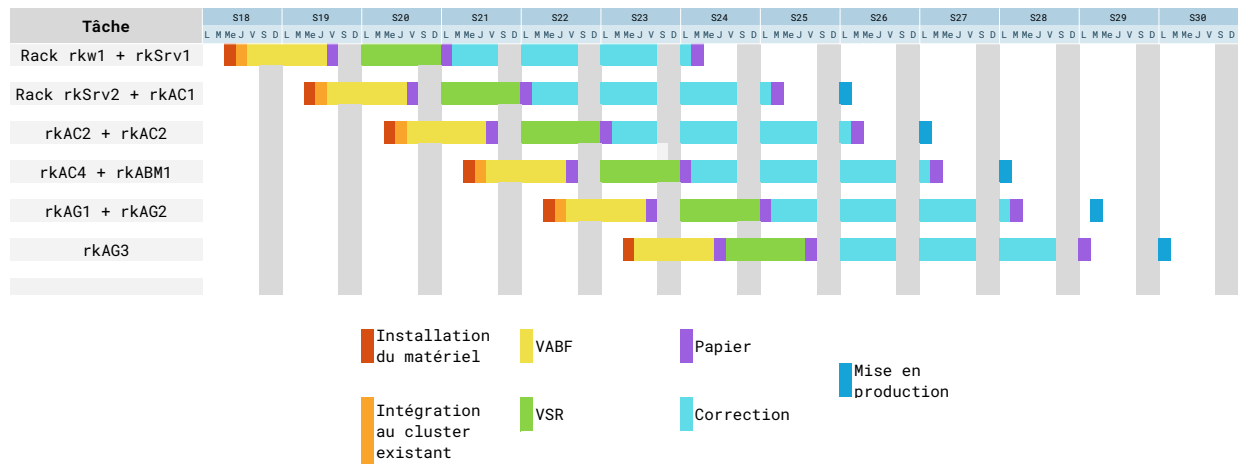


FIGURE 6 – Diagramme de Gantt de l'installation du cluster uga

- Pour rkAG3 l'installation et l'intégration sont fait dans la même journée car il n'y a qu'un seul rack livré cette semaine. -

Nous pouvons tirer de ce diagramme de Gantt les dates Clés du projet d'installation :

- 30 Avril 2025 : première reception de rack (rksw1 et rkSrv1)
- 23 Juin 2025 : Mise en production partielle (25%) de la partition AC
- 30 Juin 2025 : Mise en production de 50% de la partition AC
- 7 Juillet 2025 : Mise en production partielle de la partition AC
- 15 Juillet 2025 (car 14 férié) : Mise en production partielle de la partition AG
- 21 Juillet 2025 : Mise en production du reste de la partition AG

L'installation va donc durer 2 mois et 21 jours pour une mise en production partielle au bout de 1 mois et 23 jours. La mise en production finale aura lieu dans la période estivale et il y aura donc moins d'utilisation qu'à l'accoutumé, ce qui est très souhaitable. En effet, étant donnée que le cluster sera neuf, il se pourrait que des problèmes surviennent au début. Mais comme l'influence sera réduite, l'impact sera lui aussi amoindri. Nous tenons aussi à rappeler que ce ne sont que des estimations étant donné que nous avons fait des hypothèses sur les durées des tâches.